

INFRA Watch

Infrastructure Network for Fair Reporting and Accountability

Software Design Document (SDD)

Infrastructure Network for Fair Reporting and Accountability

Version	1.0
Date	October 2026
Prepared by	INFRA Watch Project Team
Institution	Department of Agriculture - Bureau of Agricultural and Fisheries Engineering (DA-BAFE)
Status	Baseline, reflects the implemented system
Companion document	Requirements Analysis Document (RAD)

Table of Contents

1. Introduction 7
1.1 Purpose 7
1.2 Scope 7
1.3 Intended Audience and Use 7
1.4 System Overview 8
System Objectives 8
System Boundaries 8
1.5 References 8
1.6 Relationship to Other Documents 8
1.7 Assumptions and Constraints 8
Assumptions 8
Constraints 9
2. System Overview 10
2.1 System Perspective 10
System Context Diagram 10
2.2 Design Rationale 10
2.3 System Architecture Overview 11
2.3.1 Architectural Layers 11
2.3.2 Component-Level Architecture and Code Layout 12
2.4 Major System Components 13
2.5 Inter-Component Communication 13
2.5.1 Synchronous Communication 13
2.5.2 Asynchronous Communication 13
2.6 Data Flow Overview 14
2.6.1 High-Level Data Flow 14
2.7 Design Constraints and Dependencies 14
2.8 External Interfaces 14
2.9 Performance and Scalability Considerations 15
2.10 Security and Compliance Overview 15
3. Detailed Design 16
3.1 Overview 16
Request Lifecycle 16
Shared Design Rules 16

3.2 Authentication and Access Control Service 16
3.2.1 Purpose 16
3.2.2 Key Modules and Responsibilities 17
3.2.3 Database Entities 17
3.2.4 Data Flow Example: Registration 17
3.2.5 Permission and Scope Logic 17
3.2.6 Security Measures 18
3.3 Project Data Service (ABEMIS Synchronization) 18
3.3.1 Purpose 18
3.3.2 Workflow and Service Flow 18
3.3.3 Key Modules 19
3.3.4 Status Transition Logic 19
3.3.5 Algorithm Overview 19
3.3.6 Integration Points and Failure Handling 20
3.4 Public Portal Query Layer 20
3.4.1 Purpose 20
3.4.2 Key Modules 20
3.4.3 Map and Layers 20
3.4.4 Public-Safe Output 21
3.5 Citizen Engagement Services 21
3.5.1 Feedback Pipeline 21
3.5.2 E-Report Pipeline 21
3.5.3 Upload and Moderation Pipeline 22
3.5.4 Storage Design (MinIO) 22
3.5.5 Comments, Votes, and Moderation of Comments 23
3.5.6 SLA Design 23
3.6 SMS Grievance Service 23
3.6.1 Purpose 23
3.6.2 Key Modules 23
3.6.3 Data Flow 24
3.7 Notification Service 24
3.8 Analytics and Data Quality Services 24
3.9 AI Services 25
3.9.1 Provider Abstraction 25
3.9.2 Public Assistant (ANIA) 25

3.9.3 Knowledge Base 25
3.9.4 MYDAS Dashboard Designer 26
3.9.5 Managerial AI and Voice 26
3.10 Content and Configuration Service 26
3.11 Background Jobs and Scheduler 26
3.12 Audit Service 27
4. Data Design 28
4.1 Overview 28
4.2 Database Architecture 28
4.2.1 Type and Connection 28
4.2.2 Grouping by Domain 28
4.3 Logical Data Model 28
4.4 Physical Data Model 30
4.4.1 Naming and Type Conventions 30
4.5 Key Entities and Attributes 31
4.5.1 Table `projects` 31
4.5.2 Tables `feedback`, `feedback_comments`, `feedback_comment_votes` 31
4.5.3 Tables `issues` and `issue_responses` 32
4.5.4 Tables `user`, `session`, `background_jobs`, `audit_logs` 32
4.5.5 Knowledge, Notification, and Other Tables 32
4.6 ORM and API Integration 33
4.7 Data Integrity and Validation 33
4.8 Indexing, Caching, and Optimization 33
4.9 Migrations, Backup, Archiving, and Retention 34
5. Interface Design 35
5.1 Interface Principles 35
5.2 Error and Success Response Format 35
5.3 Authentication Endpoints 35
5.4 Public Endpoints 36
5.5 Signed-In Citizen Endpoints 36
5.6 Staff Endpoints 37
5.7 Server Actions 38
5.8 External Interface Contracts 38
5.8.1 ABEMIS 38
5.8.2 SMS Gateway and Feed 39

5.8.3 Email 39
5.9 Real-Time Interfaces 39
5.10 Rate Limiting and Request Bounds 39
5.11 Pagination, Filtering, and Sorting 39
5.12 Caching Headers 40
6. Workflow Design 41
6.1 Overview 41
6.2 State Machines 41
6.3 Feedback Workflow 41
6.4 E-Report Workflow 42
6.5 SLA Monitoring and Escalation 42
6.6 SMS Grievance Workflow 42
6.7 Notification Workflow 42
6.8 Knowledge Document Workflow 43
6.9 Account and Live-Video States 43
6.10 Design Constraints 43
7. Security Design 44
7.1 Overview 44
7.2 Security Objectives 44
7.3 Security Architecture Overview 44
7.4 Authentication Design 44
7.4.1 Mechanism 44
7.4.2 Session Management 45
7.5 Authorization and Scope 45
7.5.1 Role, Resource, and Action Model 45
7.5.2 Scope Resolution 45
7.6 Data Protection 45
7.7 Upload Security 45
7.8 Security for Inter-Service and External Interfaces 46
7.9 AI Safety Design 46
7.10 Security Monitoring and Auditing 46
7.11 Privacy and Compliance (RA 10173) 46
7.12 Security Headers and Browser Protections 46
7.13 Summary of Controls and Known Gaps 47
8. Error Handling and Logging 48

8.1 Objectives 48
8.2 Error Taxonomy 48
8.3 Client Error Messages 48
8.4 Logging Design 48
8.5 Audit Entries 49
8.6 Failure Handling by Component 49
8.7 Monitoring and Alerting Recommendations 49
8.8 Example Scenarios 50
Scenario A: ABEMIS returns HTTP 526 at the scheduled hour 50
Scenario B: a citizen uploads a renamed executable as a photo 50
Scenario C: two reviewers reply to the same SMS 50
9. Deployment Design 51
9.1 Topology 51
9.2 Build and Container 51
9.3 Compose and Reverse Proxy 51
9.4 Start-Up Sequence 52
9.5 Configuration 52
9.6 Release Checklist 52
9.7 Backup and Recovery 52
10. Quality and Testing Design 53
10.1 Test Layers 53
10.2 Design for Testability 53
10.3 Acceptance 53
11. Appendices 54
Appendix A. Environment Variable Reference 54
Appendix B. Table Catalog 54
Appendix C. Glossary 56
Appendix D. Change History and Document Control 56

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes how INFRA Watch is designed and built: its architecture, components, data model, interfaces, workflows, security controls, error handling, and deployment. It is the technical companion to the Requirements Analysis Document (RAD), which states what the system must do.

The design recorded here is the design of the running system, verified against the repository in October 2026. Where the implementation has a known limitation, the limitation is noted beside the design rather than omitted, so that readers can plan around it.

The document is meant to let a new developer:

- Find where each requirement is realised in code, data, and configuration.
- Understand why the main design choices were made and what they cost.
- Change a module without breaking the rules that other modules rely on (scope, audit, public-safe output).
- Deploy, operate, and troubleshoot the system.

1.2 Scope

The SDD covers the whole INFRA Watch application: the public portal, the citizen account area, the staff console, the background jobs, and the integrations with ABEMIS, SMS, email, storage, maps, and AI providers. It does not cover ABEMIS itself, the hosting platform, or the contents of the SMS gateway.

Covered	Not covered
Web application (Next.js), server actions, route handlers	ABEMIS internals; its own RAD and SDD describe it
PostgreSQL schema, migrations, caching, queue	Operating-system, network, and Docker host administration
Integrations as consumed by INFRA Watch	The SMS gateway's and SMTP server's internals
Security, logging, and deployment design	Organizational policy and staffing

1.3 Intended Audience and Use

Audience	Use
Developers	Primary reader. Chapters 3 to 6 are the working reference.
Database administrators	Chapter 4 and Appendix B for schema, indexes, migrations, and backups.
DevOps and system administrators	Chapter 9 for deployment and Appendix A for configuration.
Security reviewers	Chapter 7 and the limitations noted throughout.
QA	Chapter 10 and the workflow rules in Chapter 6.
BAFE technical leads	Chapters 2 and 3 for an overview of how the portal works.

1.4 System Overview

System Objectives

- Present AMEFIP and INS project data from ABEMIS (fiscal years 2021 to 2026) to the public, accurately and with visible freshness.
- Collect citizen feedback, E-Reports, and SMS grievances with verifiable evidence, and route them to the right staff.
- Let BAFE respond within the 72-hour standard and show how well it is doing.
- Give managers scoped analytics and data-quality visibility.
- Protect personal data and make every staff action traceable.

System Boundaries

INFRA Watch is a read model of ABEMIS plus a participation layer. It never writes to ABEMIS and never edits project facts. It owns citizen input, staff decisions, notifications, content, and the audit trail. Figure 1 shows its place among people and external systems.

1.5 References

Reference	Use in this document
RAD for INFRA Watch (companion)	Requirement identifiers FR-* and NFR-* cited throughout
`docs/00` to `docs/13` and `docs/current-system/`	Original design notes and the implemented-feature inventory
`docs/erd/` (DBML and Mermaid)	Editable entity-relationship diagram generated from the live schema
`lib/db/schema.ts`, `auth-schema.ts`, `drizzle/`	Authoritative schema and migrations
`docs/reviews/code-security-review-2026-09-27.md`	Review findings referenced as limitations
Better Auth, Next.js, Drizzle, PostGIS, MinIO documentation	Framework behavior
RA 10173 (Data Privacy Act); WCAG 2.1 AA; PSGC	Compliance and reference standards

1.6 Relationship to Other Documents

The RAD defines each requirement; this SDD explains the design that satisfies it and cites the identifier. The ERD files and the route inventory are generated or maintained alongside the code and take precedence over any copy in this document if they differ.

1.7 Assumptions and Constraints

Assumptions

- ABEMIS exposes the infrastructure list endpoint with an API key and remains broadly available.
- PostgreSQL, MinIO, nginx, and DNS are provided and backed up outside the application.
- An SMTP relay and an SMS gateway are available for production.

- One application replica is the normal deployment; the design tolerates more with the limitations in Section 2.9.

Constraints

- Project data comes only from ABEMIS, only for fiscal years 2021 to 2026, and is read-only.
- TypeScript on Next.js 16 (App Router) with Better Auth, Drizzle ORM, and PostgreSQL; schema changes only through migrations.
- The container listens on loopback; TLS terminates at nginx.
- Citizen personal data is handled under RA 10173.

2. System Overview

2.1 System Perspective

INFRA Watch is a single deployable web application. It serves four audiences (public visitors, signed-in citizens, staff, and administrators) from one code base, and talks to a small set of external systems. All durable state lives in PostgreSQL and MinIO; the application process holds only caches and live notification connections.

System Context Diagram

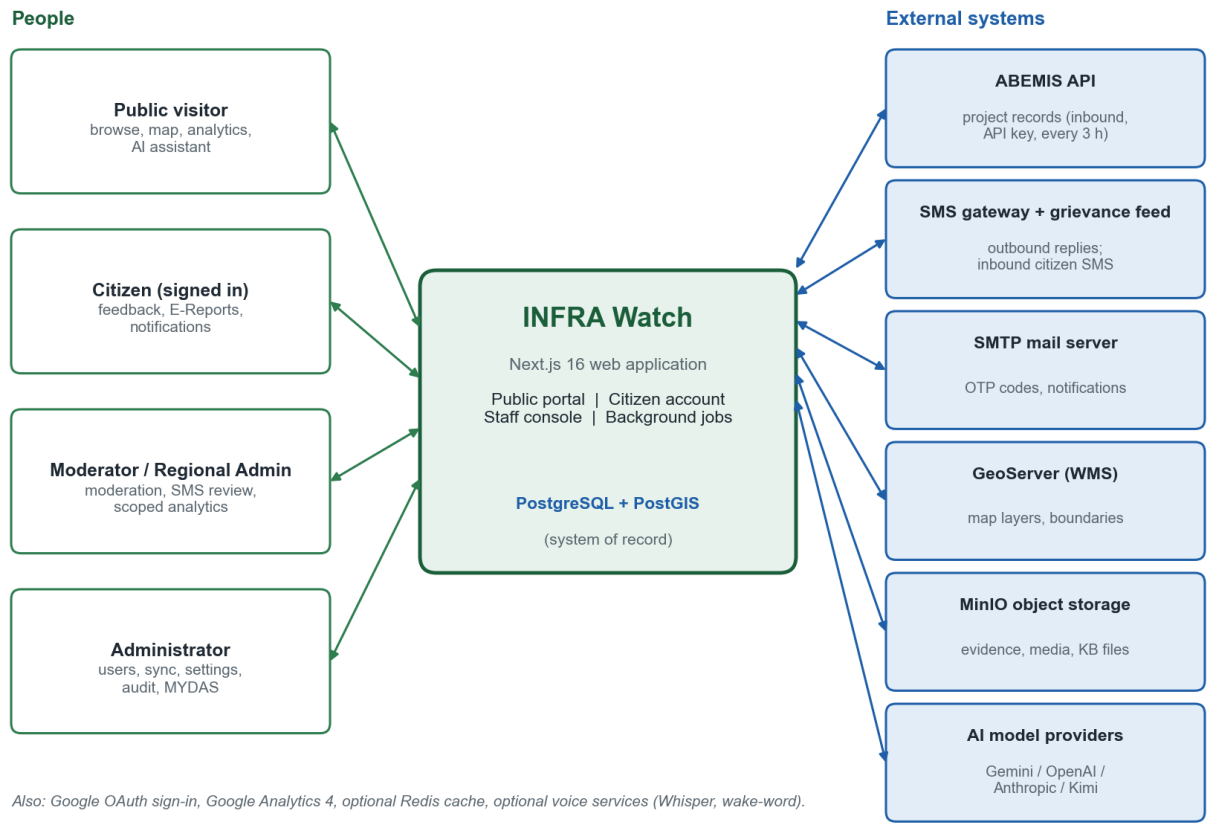


Figure 1. System context: people, the INFRA Watch application, and external systems.

2.2 Design Rationale

The main design decisions, with the reason for each and what it costs:

Decision	Reason	Trade-off
Single Next.js monolith (server components, server actions, route handlers)	One team, one deployment, shared types and validation between browser and server; simple to run on one host	Scaling is vertical first; in-process pieces (live notifications) assume one replica
Read model of ABEMIS in PostgreSQL	Fast public reads, filters, analytics, and PostGIS queries that do not depend on ABEMIS uptime	Data is only as fresh as the last sync; a mapping layer must follow ABEMIS changes

Decision	Reason	Trade-off
Better Auth with a Drizzle adapter and the admin and access-control plugins	Maintained library for sessions, OTP, OAuth, bans, and role statements	Direct plugin endpoints must be restricted so they cannot bypass app-level scope checks
Declarative permissions in `lib/permissions.ts`	One reviewable list of resource and action per role	Scope rules (region, program) are separate and must be applied in every query
PostgreSQL job queue with leases instead of an external broker	No extra infrastructure; jobs survive restarts; dedupe and retries are simple SQL	Throughput is modest; polling worker lives in the app process
`node-cron` inside the app for schedules	Zero extra services; schedules start with the app	Every replica would run the schedule; dedupe keys and 20-hour skip rule contain the effect
Embeddings stored as JSON and ranked in the application	No `pgvector` dependency; small knowledge base	Linear scan; not suitable for very large corpora
Two MinIO buckets (public media, private evidence and knowledge files)	Public images can be served directly; sensitive files only through authenticated handlers	Start-up must reconcile bucket policies and fail closed
AI through bounded, read-only tools with a refusal policy	Answers grounded in portal data; limited blast radius	The assistant cannot answer anything the tools cannot fetch
Recommendation-only data quality	ABEMIS stays the system of record	Fixes happen elsewhere and arrive on the next sync
Auto-accept setting for feedback and issues (default on)	Citizens see immediate acknowledgment; staff review after the fact	Content is public after automated checks only; staff can switch it off

2.3 System Architecture Overview

2.3.1 Architectural Layers

The system is a layered monolith. Each layer talks only to the one below it. Figure 2 shows the layers and the components in each.

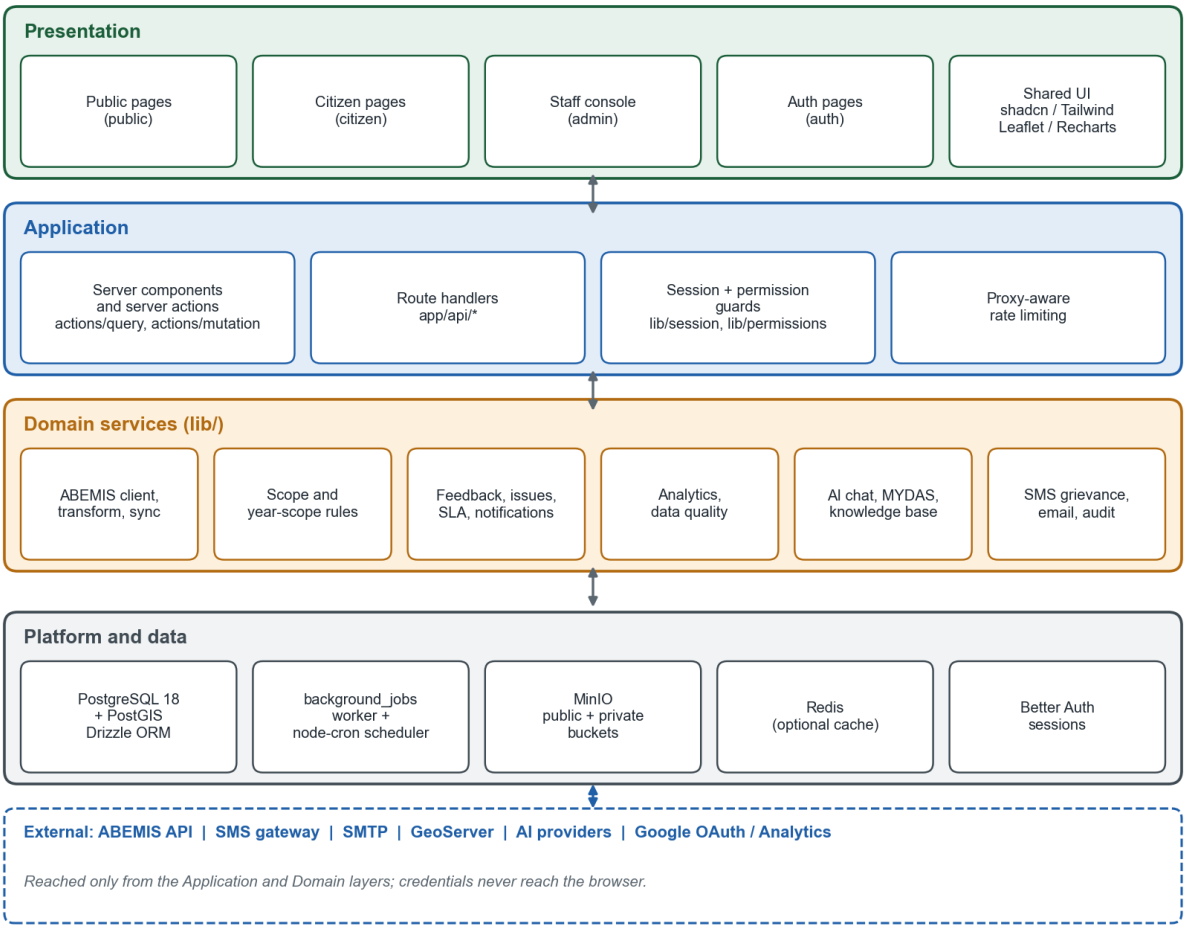


Figure 2. Architectural layers and components.

2.3.2 Component-Level Architecture and Code Layout

Path	Contents
`app/(public)`, `(citizen)`, `(auth)`, `(admin)`, `(protected)`	Route groups for the four audiences; `(admin)/layout.tsx` requires a staff role
`app/api/*`	Route handlers (JSON, uploads, server-sent events, AI, health)
`actions/query`, `actions/mutation`	Server actions that read and write data, one file per domain (for example `public-projects.query.ts`, `feedback.mutation.ts`)
`lib/`	Domain services: `abemis/`, `analytics/`, `data-quality/`, `sms-grievance/`, `mydas/`, `voice/`, `public-analytics/`, plus auth, scope, audit, cache, minio, scheduler, jobs, notifications, chat, knowledge base
`lib/db`, `auth-schema.ts`, `drizzle/`	Drizzle schema, connection, and 39 SQL migrations
`components/`, `hooks/`, `providers/`, `i18n/`	UI components, client hooks, context providers, translations
`constants/`, `types/`	Stage mapping, program list, shared types and Zod schemas
`scripts/`, `docker/`, `deploy/`	Migration runner, sync scripts, container start script, nginx sample

2.4 Major System Components

Component	Responsibility	Requirements
Authentication and access	Accounts, OTP, OAuth, sessions, role and permission checks, moderator scope	FR-AUTH-*
Project data service	ABEMIS client, transform, scope filter, upsert, snapshots, location sync	FR-SYNC-*
Public query layer	Directory, record, map, analytics, export with public-safe output and caching	FR-PUB-*
Engagement services	Feedback, comments, votes, E-Reports, uploads, moderation, evidence	FR-FB-, *FR-ER-*, FR-UP-01, FR-GEO-01
SMS grievance service	Feed intake, review store, replies, auto-link and auto-acknowledge	FR-SMS-01
Notification service	Persistence, live stream, email and SMS delivery, SLA reminders	FR-NOT-*
Analytics and data quality	Managerial dashboard, snapshots, schedule health, findings, engagement events	FR-ADM-01..04, 09
AI services	Chat, knowledge base, managerial AI, MYDAS, voice	FR-AI-*, FR-ADM-02
Content and configuration	Articles, announcements, templates, layers, live videos, settings	FR-PUB-05, FR-ADM-07
Jobs and scheduler	Queue, worker, cron schedules, startup tasks	FR-SYNC-01, FR-NOT-01
Audit and health	Audit log writer and reader, service health	FR-ADM-06, 08

2.5 Inter-Component Communication

2.5.1 Synchronous Communication

- **Browser to server:** page requests render server components; forms call server actions; client code calls route handlers with JSON or multipart bodies.
- **Server to PostgreSQL:** Drizzle over a pooled `postgres` connection (default 10 connections, 20 s idle timeout, 30 min lifetime).
- **Server to external services:** HTTPS from server code only (ABEMIS, SMS, SMTP, AI providers, MinIO); the browser talks directly only to GeoServer (WMS tiles) and Google Analytics.

2.5.2 Asynchronous Communication

- **Job queue:** `background_jobs` rows with kinds `abemis-sync`, `location-sync`, `media-moderation`, and `notification-delivery`. A worker claims a job with a 2-minute lease, extends it with a heartbeat, retries up to 3 times with exponential back-off (60 s doubling, capped at 15 minutes), and reclaims jobs whose lease expired.
- **Cron schedules** (Asia/Manila): ABEMIS sync every 3 hours (enqueues a job); chat-history purge 03:15; engagement-analytics maintenance 03:30; SLA reminders every 15 minutes; SMS auto-link and auto-acknowledge every minute.

- **Server-sent events:** `/api/notifications/stream` for a signed-in user and `/api/admin/jobs/[id]/stream` for job progress.

2.6 Data Flow Overview

2.6.1 High-Level Data Flow

1. **Ingest:** ABEMIS to `projects` (via the sync job), with a run record in `sync_logs` and a daily row per project in `project_metric_snapshots`.
2. **Publish:** public pages read `projects` through query functions that apply the fiscal-year scope and strip non-public fields; frequent reads are cached.
3. **Participate:** a citizen submits feedback or an E-Report; evidence goes to MinIO; the record goes to PostgreSQL; staff and the citizen are notified.
4. **Respond:** scoped staff moderate or respond; every action writes `audit_logs`; the citizen sees the outcome in their account and, when public, on the portal.
5. **Measure:** SLA reports and the managerial dashboard aggregate the same tables; snapshots give trends.

2.7 Design Constraints and Dependencies

Constraint or dependency	Design consequence
ABEMIS is read-only and sometimes unavailable	Frequent schedule with skip rule; freshness always shown; no write-back
Fiscal-year scope 2021-2026	One shared condition (`projectYearScopeCondition`) used in sync and every query
RA 10173	Anonymity in public DTOs; contact data staff-only; retention limits; deletion pages
Single host, external data services	Application is stateless except caches and live connections; configuration by environment
Better Auth plugin model	Roles and statements declared in one file; app-level guards still required for scope

2.8 External Interfaces

System	Interface summary	Details
ABEMIS	HTTPS GET `/api/infra-amefip-list`, API key, pages of 500	Section 5.8.1
SMS gateway and feed	HTTPS POST (send); HTTPS GET (inbound feed)	Section 5.8.2
SMTP	Nodemailer, STARTTLS	Section 5.8.3
MinIO	S3-compatible, two buckets	Section 3.5.4
GeoServer	WMS layers from `map_layers`; KML proxy	Section 3.4.3
AI providers	Vercel AI SDK; provider chosen by `AI_PROVIDER`	Section 3.9

2.9 Performance and Scalability Considerations

- **Caching:** `cachedPublic(key, ttl, load)` keeps values in a bounded in-process map (at most 100 entries, 30 seconds maximum) and in Redis when configured (TTL per call), de-duplicates concurrent loads, namespaces keys per instance, and can be cleared by an administrator. Public analytics views also send `Cache-Control: public, max-age=60, stale-while-revalidate=300`.
- **Bounded work:** admin lists clamp page size to 10 to 100; analytics refuse more than 30,000 rows; chat requests are limited to 64 KB; JSON bodies to 32 KB by default.
- **Indexes:** project identifier, province, region, status, and funding year; issue status, project, and ticket; feedback status, project, and sentiment; job claim and dedupe.
- **Scaling:** a second replica works for reads and writes. Two limits apply: live notifications are fanned out in-process (stored history stays complete), and every replica would start the cron schedules (jobs are deduplicated, and the sync skips if a recent success exists).

2.10 Security and Compliance Overview

Security is layered: TLS and header handling at nginx; sessions, roles, and scope in the application; strict validation and moderation of uploads; public-safe output objects; read-only, bounded AI tools; an audit log of staff and security events; and secrets only in the environment. Chapter 7 gives the details. Compliance targets are RA 10173 for personal data and WCAG 2.1 AA for accessibility.

3. Detailed Design

3.1 Overview

This chapter describes each service: its purpose, its main modules, the data it uses, a representative flow, and the safeguards it relies on. All services are modules of one application, not separate processes; "service" means a cohesive group of code under `lib/` and `actions/`.

Request Lifecycle

1. The browser sends HTTPS to nginx, which forwards to the application on loopback.
2. A page render or action resolves the session (`getSession`, cached per request) and applies the guard for that area: `requireAuth`, `requireAdminOrModerator`, `requireAdmin`, `requireAdminOrRegionalAdmin`, or, for APIs, `requireStaffApi`.
3. Mutating staff requests also call `assertMutationOrigin`; JSON bodies go through `jsonBody` (content type, origin, and size checks).
4. The action calls `requirePermission(role, resource, action)` and, for project-linked data, adds the scope conditions to its query.
5. Input is validated with Zod; the domain function runs against PostgreSQL through Drizzle.
6. Significant changes call `logAudit`; notifications are persisted and queued.
7. Output is converted to a public-safe or role-appropriate object; errors go through `apiError`, which never leaks internals.

Shared Design Rules

- **Scope first:** any query that touches projects, feedback, or issues for staff includes `getProjectScopeConditions(user)`.
- **Public-safe output:** public endpoints return purpose-built objects (for example `lib/public-issue-dto.ts`), never raw rows.
- **Audit by default:** every staff mutation and security event writes `audit_logs`.
- **Fail closed:** unknown role, missing scope, failed policy check, or private-bucket anomaly denies access.
- **Explicit unknowns:** missing data is returned as unavailable, never as zero or a guess.

3.2 Authentication and Access Control Service

3.2.1 Purpose

Identify the user, hold a session, and decide what the user may do and which projects they may see (FR-AUTH-01 to 06).

3.2.2 Key Modules and Responsibilities

Module	Responsibility
`lib/auth.ts`	Better Auth instance: Drizzle adapter, email and password with required verification, `emailOTP` plugin (6 digits, `EMAIL_OTP_EXPIRY`, sign-up disabled), optional Google provider, admin plugin with role statements, trusted origins, user additional fields (`role`, `region`, `assignedAgency`, `phoneNumber`, ban fields), bootstrap-admin hook
`lib/permissions.ts`	Resource and action statements; roles `citizen`, `moderator`, `regional_admin`, `admin`; `hasPermission`, `requirePermission`
`lib/session.ts`	Cached `getSession`; guards `requireAuth`, `requireAdmin`, `requireAdminOrModerator`, `requireAdminOrRegionalAdmin`; `canAccessAdmin`
`lib/scope.ts`, `lib/moderator-scope.ts`	Project scope conditions, `checkModeratorScope`, assignment normalization
`lib/staff-api.ts`	`requireStaffApi`, `assertMutationOrigin`, `jsonBody`, `apiError` for route handlers
`lib/auth-otp-email.ts`	Delivery of verification and reset codes through SMTP
`actions/mutation/users.mutation.ts`	Administrator operations on users with self-protection and audit

3.2.3 Database Entities

`user`, `session`, `account`, `verification` (Better Auth, defined in `auth-schema.ts`), `account_preferences`, `user_tour_progress`, `audit_logs`.

3.2.4 Data Flow Example: Registration

1. The visitor submits name, email, and password on `/sign-up`.
2. Better Auth runs the `user.create.before` hook, which assigns the role `citizen` (or `admin` for a configured bootstrap address), then inserts `user` and `account`.
3. The OTP plugin creates a 6-digit code in `verification` and calls `sendVerificationOTP`; in production `deliverAuthCode` sends it by SMTP, in development it is logged.
4. The visitor enters the code; the user is marked verified and, because `autoSignInAfterVerification` is on, a `session` is created.

3.2.5 Permission and Scope Logic

Permissions are looked up in the role's statement list; scope is a query-level filter. Both excerpts are simplified from the code.

```
// lib/permissions.ts
export function hasPermission(role, resource, action) {
  const statements = roleRegistry[role]?.statements;
  return Boolean(statements?.[resource]?.includes(action));
}
export function requirePermission(role, resource, action) {
  if (!hasPermission(role, resource, action))
    throw Object.assign(new Error("You do not have permission..."), { status: 403 });
}

// lib/scope.ts
export function getProjectScopeConditions(user) {
  const conditions = [projectYearScopeCondition()];        // always: FY 2021-2026
  if (user.role !== "moderator") return conditions;         // admin, regional_admin
  if (!hasAssignedModeratorScope(user)) return [sql`false`]; // fail closed
```

```
  if (region) conditions.push(ilike(projects.region, region));
  if (agency) conditions.push(eq(projects.program, agency));
  return conditions;
}
```

3.2.6 Security Measures

- Secrets and public URL are mandatory in production; the app throws at start-up without them.
- Origins are limited to a trusted list; staff mutations reject cross-origin requests.
- Bans are enforced by the admin plugin; administrators can end every session of a user.
- Self-protection (no self-ban, self-delete, self-role-change, or self-session-termination) is enforced on the server.
- Known limitation: Better Auth's own administrative endpoints are reachable by authenticated admins and regional admins; the review of 27 September 2026 recommended restricting direct plugin administration so regional-admin region limits cannot be bypassed. Confirm this is applied.

3.3 Project Data Service (ABEMIS Synchronization)

3.3.1 Purpose

Keep the `projects` read model current and inside scope, and record how the data changes over time (FR-SYNC-01 to 05).

3.3.2 Workflow and Service Flow

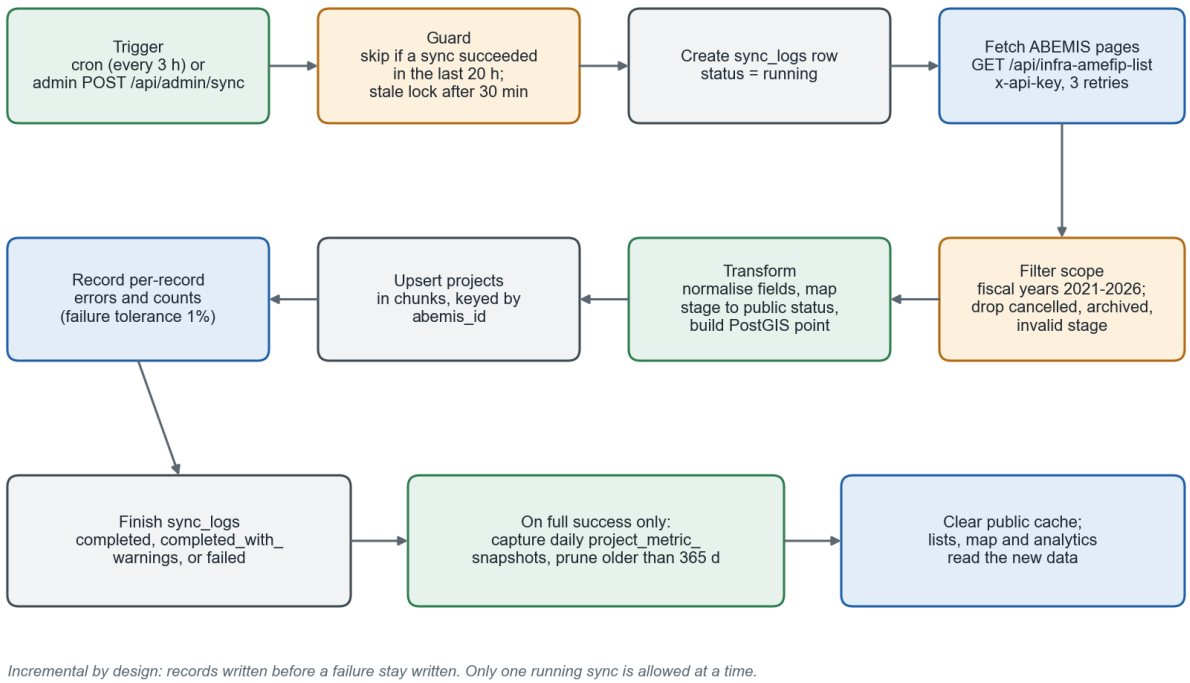


Figure 3. ABEMIS synchronization flow.

3.3.3 Key Modules

Module	Responsibility
`lib/scheduler.ts`	Cron schedule (default `0 */3 * * *`, Asia/Manila) that skips when a sync succeeded recently and otherwise enqueues an `abemis-sync` job
`lib/background-worker.ts`	Claims jobs, runs `abemis-sync` and `location-sync`, reports progress, marks a run failed if a worker restart interrupted it
`lib/abemis/client.ts`	`fetchInfraProjects` with `x-api-key`, page and `page_size`, up to 3 retries with back-off; `buildUrl` keeps requests on the configured origin; redirects are refused
`lib/abemis/transform.ts`	`isInfraWatchProject` (project-type filter) and `transformAbemisProject` (normalizes fields, infers progress, builds location and point)
`lib/abemis/year-scope.ts`	`ABEMIS_SYNC_START_YEAR` 2021, `ABEMIS_SYNC_END_YEAR` 2026, exclusion reasons, `projectYearScopeCondition`
`lib/abemis/sync.ts`	Runs a sync: active-run guard, paging (500), scope filter, chunked upsert, per-record error capture, status decision, snapshot capture
`lib/analytics/project-metric-snapshots.ts`	`captureProjectMetricSnapshots`, `pruneProjectMetricSnapshots` (365 days)
`constants/stage-mapping.ts`	Raw stage to public status mapping

3.3.4 Status Transition Logic

ABEMIS stage text is mapped to three public statuses. Unrecognised text uses a keyword fallback (complete or done gives Completed; ongoing, progress, or an implementation stage that is not "ready" gives On going) and otherwise Not yet started.

ABEMIS stage or status	Public status
Proposal, Pre-implementation, Procurement, Under-Procurement, Planned, Proposal Validated, Implementation-ready, For Review, Incomplete Documents, Not yet started	Not yet started
Implementation, Ongoing, Under-Construction, For Turn-Over, Suspended	On going
Completed, Inventory	Completed

3.3.5 Algorithm Overview

```
runSync(trigger):
  if hasActiveSync(): return                      // running and younger than 30 min
  log = insert sync_logs(status = 'running')
  for page in 1..totalPages (page_size 500, retries 3):
    for record in page:
      reason = getAbemisSyncExclusionReason(record)   // year, stage, cancelled, invalid
      if reason: counts[reason]++; continue
      row = transformAbemisProject(record)
      collect row
    upsert collected rows in chunks ON CONFLICT (abemis_id)
  status = failureRate < 1% ? 'completed_with_warnings' : 'failed'  (or 'completed')
  update sync_logs(status, counts, errors, duration)
  if status == 'completed': captureProjectMetricSnapshots(); prune(365 days)
  clearPublicCache()
```

3.3.6 Integration Points and Failure Handling

Failure	Behavior
ABEMIS returns an error or times out	Retried up to 3 times; the run fails and is logged; the next tick (3 h) tries again
Worker restarts mid-run	The run is marked failed ("will be retried"); the job lease expires and the job is reclaimed
A few records fail	Counted in `records_failed`; below 1% of processed records the run is *completed with warnings*
Two runs overlap	Prevented by the active-run guard and the job dedupe key
Source ID corrected upstream	A fallback path may delete and recreate a project when foreign keys block renaming; this can cascade to dependent rows (limitation R-03 in the RAD)

3.4 Public Portal Query Layer

3.4.1 Purpose

Turn the `projects` read model into fast, safe public pages and APIs (FR-PUB-01 to 07).

3.4.2 Key Modules

Module	Responsibility
`actions/query/public-projects.query.ts`	`getPublicProjects`, `getPublicMapPins`, `getPublicMapProjectDetails`; fiscal-year scope and public-status filter applied
`lib/public-project-directory.ts`	Parses and bounds directory state from the URL (search, filters, sort, page)
`lib/public-project-record.ts`, `lib/public-source-media.ts`	Builds the project record; sanitizes source media and geotags
`lib/public-project-map.ts`	Marker model; keeps only coordinates inside Philippine bounds (`lib/philippine-coordinates.ts`)
`lib/public-analytics/*`	`getPublicProjectSnapshot`, filters, `VIEW_BUILDERS` for `/api/public/[view]`, stage condition shared with sync
`actions/query/analytics.query.ts`	Portfolio analytics with a 30,000-row ceiling and explicit unavailable state
`lib/cache.ts`	`cachedPublic`, `clearPublicCache`, optional Redis client
`lib/project-export.ts`	CSV and JSON export of the filtered list

3.4.3 Map and Layers

- Markers come from PostGIS-backed coordinates and are clustered in the browser (`react-leaflet-cluster`).
- Overlays are rows in `map_layers` (GeoServer URL, workspace, layer, style, opacity, legend, order); `NEXT_PUBLIC_GEOSERVER_URL` and workspace are inlined at build time, and an empty URL hides all overlays.
- `/api/kml-proxy` fetches imported KML for the browser with a 10 MB limit and server-side URL checks.

3.4.4 Public-Safe Output

Public responses are built from allow-listed fields. Source media and geotags pass through a sanitizer; coordinates are dropped when unusable; feedback is limited to approved items; E-Report records pass through `public-issue-dto.ts`, which removes contact data, internal notes, internal-only responses, and unapproved text.

3.5 Citizen Engagement Services

3.5.1 Feedback Pipeline

1. `POST /api/projects/[id]/feedback` requires a session (401 otherwise) and an existing project (404).
2. The comment is trimmed and screened with `assertCleanText`; the server enforces 1 to 1,000 characters; media references are parsed and must match the portal's own upload paths.
3. `resolveInitialFeedbackStatus()` reads the `auto_reply_config` setting. If feedback auto-accept is on (the default) the row is inserted as `approved`; otherwise as `pending`.
4. The row is inserted with `userId`, rating, category, anonymity flag, and media; an audit record is written.
5. Scoped staff are notified; with auto-accept on, the citizen also receives the configured acknowledgment notification from a system actor.

Owners edit or delete their own feedback through `PATCH` and `DELETE` on `/api/projects/[id]/feedback/[feedbackId]`. Staff moderation uses `PATCH /api/admin/feedback/[feedbackId]/moderate` (approve or reject with an optional note) and `DELETE /api/admin/feedback/[feedbackId]`.

3.5.2 E-Report Pipeline

1. `POST /api/issues` accepts JSON or multipart form data. Required: issue category and a description (at least 20 characters on the JSON path, 10 on the form-data path); identified reports also need a name and contact number.
2. Evidence is normalized into a canonical list (type, path, name, optional GPS track) of at most the allowed count; coordinates are validated against latitude and longitude ranges; a geo-video track is bounded in points and time.
3. `resolveInitialIssueStatus()` uses the same auto-accept setting: on gives `reviewing` with the description copied to the public description (if at least 20 characters) and an automated first response from `InfraWatch Automated System`; off gives `pending`.
4. A ticket number of the form `INFRA-<year>-<six digits>` is generated and the row inserted with `reporterUserId` when a session exists; audit and staff notifications follow.

Staff handling: `POST /api/admin/issues/[id]/responses` inserts a public or internal response, may change status, may approve the public description, sets `resolved_at` on `resolved`, writes audit, and notifies the reporter. Public reads use `GET /api/issues` and `/api/issues/[id]` through the public DTO; owners use `/api/my-issues`.

3.5.3 Upload and Moderation Pipeline

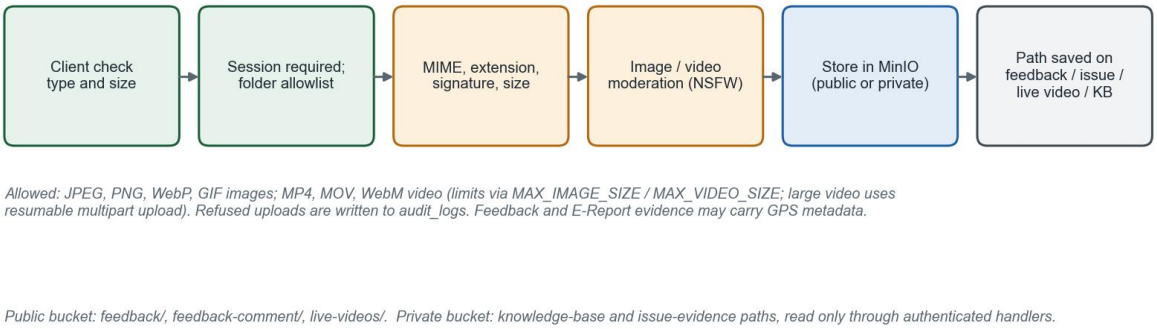


Figure 4. Evidence upload pipeline.

- `POST /api/upload?folder=...` requires a session and checks the folder against an allowlist (invalid folders are refused and audited).
- The server validates MIME type, extension, file signature, and size; the image type and size limits come from `ALLOWED_IMAGE_TYPES`, `MAX_IMAGE_SIZE`, `ALLOWED_VIDEO_TYPES`, and `MAX_VIDEO_SIZE`.
- `assertSafeImageUpload` runs NSFWJS on images (thresholds `NSFW_*`, optional fail-closed); stored videos are sampled with ffmpeg and each frame is checked (`video-moderation.ts`).
- Large video uses `/api/upload/multipart` with 50 MB parts and a resumable `upload_sessions` record; the limit for a direct upload is 20 GB.
- Previews are served by `/api/upload/preview` under an image policy that restricts what can be rendered; for private paths the handler first checks who is asking (see Section 3.5.4).
- Issue evidence is private by default. `PATCH /api/admin/issues/[id]/evidence/[index]` lets staff approve or withdraw a single image for public display, and an image only becomes visible once the issue has a published public summary.

3.5.4 Storage Design (MinIO)

Bucket	Prefixes	Access
Public (`MINIO_BUCKET_NAME`)	`feedback/`, `feedback-comment/`, `live-videos/`, other approved public media	Anonymous read limited to these prefixes by bucket policy
Private (`MINIO_PRIVATE_BUCKET_NAME`)	knowledge-base files and issue evidence	No anonymous access; read only through authenticated handlers: `/api/upload/preview` (knowledge-base readers, or the issue owner, an administrator, or an in-scope moderator for issue evidence), `/api/knowledge-base/files/[id]`, and `/api/issues/[id]/public-evidence/[index]` for evidence staff approved for public display

At start-up (production only) `ensureStorageBuckets` creates both buckets, narrows the public bucket policy to the approved prefixes, and refuses to continue if the buckets collide or if the private bucket contains an anonymous `Allow`. If MinIO cannot be reached during an upload, the code may write to a local `.storage` directory, which is not durable in a container (limitation R-04).

3.5.5 Comments, Votes, and Moderation of Comments

`feedback_comments` carries `status`, `auto_rejected`, and `rejection_reason`; text is screened with the same profanity check, and flagged comments are quarantined for the `/comment-moderation` page (`GET` and `PATCH /api/admin/comments`). Votes are one row per user and comment in `feedback_comment_votes` with a type; counters are kept on the comment.

3.5.6 SLA Design

Constant	Value	Meaning
`SLA_BREACH_HOURS`	72	Agency must report back on action taken (8888 standard)
`SLA_GENTLE_REMINDER_HOURS`	24	Nudge the scoped moderator(s)
`SLA_URGENT_REMINDER_HOURS`	60	Moderators and admins
Breach notice	after 72	Admins only

`runSlaFollowUpReminders` runs every 15 minutes and stamps `sla_gentle_reminder_at`, `sla_urgent_reminder_at`, and `sla_breach_notified_at` on `issues` and `feedback`. `actions/query/reports.query.ts` uses the same constants for the SLA reports.

3.6 SMS Grievance Service

3.6.1 Purpose

Give staff a reviewed, auditable path from a citizen's SMS to a project and a reply (FR-SMS-01).

3.6.2 Key Modules

Module	Responsibility
`lib/sms-grievance/live-source.ts`	Fetches raw messages from `SMS_GRIEVANCE_API_URL`, validates shape, maps each to an untriaged record in `needs_relevance_review`
`lib/sms-grievance/review-store.ts`	Reads and writes `sms_grievance_reviews` with a `version` check (optimistic concurrency)
`lib/sms-grievance/live-review.ts`, `policy.ts`, `issue-management.ts`	Overlay of saved decisions on the feed; rules for tagging, out-of-scope, and follow-up linking
`lib/sms-grievance/location.ts`, `categories.ts`	Parses coordinates from text; category list shared with the E-Report issue types plus SMS-only categories
`lib/sms-grievance/auto-acknowledge.ts`	Per-minute jobs: auto-link a sender's new message to their open case (within a week, never sends SMS) and optionally send a first reply
`lib/sms.ts`	`sendSMS`: normalizes the number to `+639XXXXXXXXX` and posts to the gateway
`actions/mutation/sms-grievance.mutation.ts`	Server actions for tag, out-of-scope, link, and reply; each saves first, then sends

3.6.3 Data Flow

1. The review page loads the feed server-side and overlays saved state from `sms_grievance_reviews`; if the feed is down, labelled synthetic records are shown.
2. Staff choose an actual BAFE project from the project read model; parsed text is only a hint.
3. The action writes the new state with `version + 1`; if the stored version differs, the write is rejected so two reviewers cannot send the same reply.
4. The SMS is sent after the decision is saved; the thread entry records success or failure. Internal notes are never sent.

The data model keeps the original message text unchanged; project, location, category, responsible office, region, and assignee are separate fields. Limitation: reviews do not yet create `issues` rows.

3.7 Notification Service

- **Persistence:** `persistNotification` inserts one `notifications` row and one `notification_recipients` row per user inside a transaction; duplicates are removed.
- **Recipients:** chosen from project and issue scope (moderators for the region and program, administrators for escalation, the reporter or owner); `notificationRecipientsForChannel` filters by each user's `account_preferences`.
- **Live delivery:** `realtime-notifications.ts` keeps a registry of open event-stream connections per user in the Node process and writes each new notification to the connected user's stream.
- **External delivery:** `notification-delivery` jobs send email (SMTP) and SMS; failed jobs are retried by the queue.
- **Read state:** `POST /api/notifications` marks items read; `notification_recipients.read_at` and `notification_reads` persist it.
- **Templates:** `email_templates` (slug, subject, body, variables, active) edited in `/email-templates`, with a built-in registry as fallback.

3.8 Analytics and Data Quality Services

Module	Responsibility
`lib/analytics/managerial-dashboard-query.ts`, `dashboard-filters.ts`	Strict filter schema; scope and row limits; KPIs, region and funding-year views, priority and delayed projects
`schedule-health.ts`, `completion-forecast.ts`, `dashboard-changes.ts`	Schedule-health classification from dates and progress; forecast; change detection between snapshots
`project-metric-snapshots.ts`	Daily snapshots used for trends and progress variance
`lib/data-quality/project-quality.ts`, `service.ts`	Detects missing, inconsistent, stale, or implausible source fields; paginated findings; export
`lib/analytics/citizen-event-*.ts`	Privacy-minimal engagement events: policy, rate limit, coarse network-region lookup, store, daily aggregation, retention
`actions/query/reports.query.ts`	SLA reports for issues and feedback (admin only)
`app/api/admin/analytics/*`	JSON endpoints for dashboard breakdown, drill-through, progress curve, and cost by region

Design rule: every analytic returns a state (ready, empty, unavailable, or cannot be assessed) with the reason; derived values never silently default to zero. Data-quality findings are read-only by design, and the project `PATCH` endpoint for data quality returns 405.

3.9 AI Services

3.9.1 Provider Abstraction

`lib/ai-provider.ts` maps `AI_PROVIDER` to a model through the Vercel AI SDK: Google Gemini (default, `gemini-flash-latest`), OpenAI (`gpt-4o-mini`), Anthropic, or Kimi. `AI_MODEL` overrides the model. All AI features use this one function, so provider changes are a configuration change.

3.9.2 Public Assistant (ANIA)

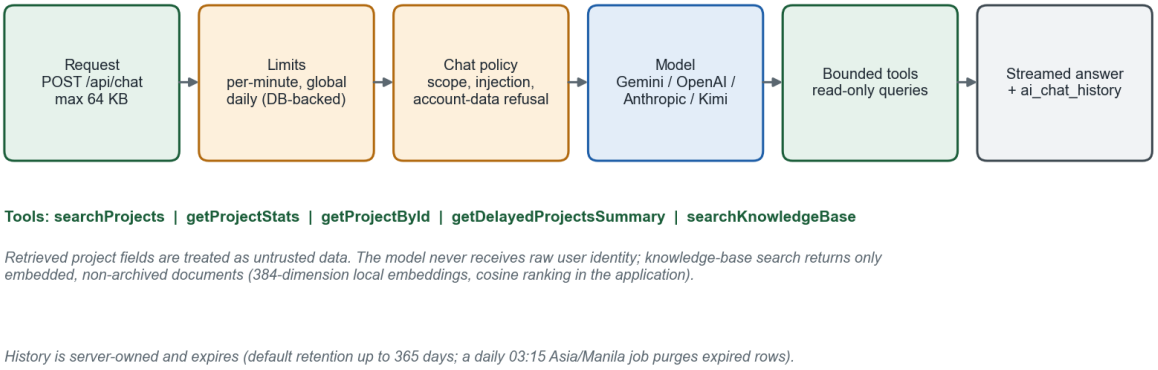


Figure 5. Public assistant request flow.

Module	Responsibility
`app/api/chat/route.ts`	Orchestrates the request; builds the system prompt including the scope instruction and the fixed list of suggested next-step keys
`lib/chat-request.ts`	Limits: 64 KB body, 20 history messages, 4,000 characters per message, 20,000 characters of history
`lib/chat-policy.ts`	Refuses unrelated or coding requests, attempts to reveal internals or user data, and prompt-injection phrases
`lib/chat-rate-limit.ts`	Per-identity per-minute limit (default 5, configurable 1 to 100) and global daily limit (default 2,000) stored in `ai_chat_rate_limits`; identity is hashed
`lib/chat-tools.ts`	Five read-only tools: `searchProjects`, `getProjectStats`, `getProjectById`, `getDelayedProjectsSummary`, `searchKnowledgeBase`
`lib/chat-history.ts`, `chat-history-*.ts`	Server-owned history with owner key, status, token counts, expiry, and daily purge
`lib/chat-grounding.ts`, `chat-links.ts`, `chat-actions.ts`	Grounding checks; safe links and action targets limited to known keys

3.9.3 Knowledge Base

1. **Upload or FAQ entry:** staff upload PDF, TXT, or Markdown, or type a question and answer; the file goes to the private bucket and a `kb_documents` row is created.
2. **Extract:** `kb-extractor.ts` reads PDF (pdf-parse), TXT, or Markdown, normalizes Unicode and whitespace, and rejects empty text.

3. **Chunk:** `kb-chunker.ts` splits text (defaults `KB_CHUNK_SIZE` 500, `KB_CHUNK_OVERLAP` 50).
4. **Embed:** `kb-embedding.ts` runs the `all-MiniLM-L6-v2` model locally (Transformers.js), producing 384-dimension normalized vectors.
5. **Store:** chunks are written to `kb_chunks.embedding` as JSON, in a transaction that swaps old chunks for new ones when reindexing.
6. **Search:** `kb-search.ts` embeds the query, loads active chunks of embedded documents, and ranks them by cosine similarity in the application.

3.9.4 MYDAS Dashboard Designer

MYDAS (admin only) turns a plain-language question into a chart. The model proposes a query and a display type from `analytics-catalog.ts`; `sql-guardrails.ts` then enforces: one statement; no comments; must start with `SELECT`; forbidden keywords (data-changing, administrative, file, and `INTO` or `WITH` constructs); only the tables `projects`, `feedback`, `issues`, `contact_messages`; a row limit (default 500, maximum 1,000); and execution inside a transaction with `statement_timeout` of 5 seconds. The prompt only describes selected columns, but the validator limits tables, not columns, so access stays admin-only.

3.9.5 Managerial AI and Voice

- **Managerial copilot and executive brief** (`ENABLE_MANAGERIAL_AI=true`): `managerial-ai-policy.ts`, `managerial-ai-prompt.ts`, and `managerial-ai-tools.ts` bound the context to the user's authorized dashboard scope; output keeps its verification notice.
- **Voice** (`VOICE_ASSISTANT_ENABLED`): `/api/voice/wake-token` issues a signed token (`WAKE_WORD_TOKEN_SECRET`); `/api/voice/transcribe` forwards bounded audio to a private Whisper-compatible service (`LOCAL_WHISPER_URL`); speech is synthesized in the browser with Kokoro or a browser fallback.

3.10 Content and Configuration Service

`lib/content-management.ts` and `actions/mutation/content.mutation.ts` manage articles, announcements, email templates, and map layers; `live-videos.mutation.ts` manages broadcasts inside a transaction that guarantees at most one live and one featured entry; `settings.mutation.ts` writes `system_settings`. Settings include `auto_reply_config` (auto-accept toggles and messages), `maintenance_mode`, and `public_banner`. While maintenance mode is on, the public sees a notice and staff (admin, regional admin, moderator) keep access; `/sign-in` and `/forget-password` stay reachable.

3.11 Background Jobs and Scheduler

Task	Trigger	Mechanism	Function
ABEMIS sync	Cron every 3 h; manual	Queue job `abemis-sync`	Refresh projects and snapshots
Location sync	Manual	Queue job `location-sync`	Refresh PSGC locations
Media moderation	On demand	Queue job `media-moderation`	Check stored media
Notification delivery	On event	Queue job `notification-delivery`	Email and SMS
Chat history purge	Daily 03:15	Cron	Delete expired chat rows
Engagement maintenance	Daily 03:30	Cron	Prune and aggregate engagement events

Task	Trigger	Mechanism	Function
SLA reminders	Every 15 min	Cron	Gentle, urgent, and breach notices
SMS auto-link and acknowledge	Every minute	Cron (no-op unless configured)	Link follow-ups; first SMS reply
Upload clean-up	Periodic in worker	Worker	Remove expired upload sessions

The scheduler starts when `ENABLE_SCHEDULER=true`, or in production unless `ENABLE_SCHEDULER=false`. The worker starts in production unless `ENABLE_BACKGROUND_WORKER=false`, and in development only when it is `true`. In production the storage policy reconciliation completes before the scheduler starts. An administrator can list jobs, enqueue an `abemis-sync` or `location-sync` job (HTTP 202), and retry a failed one through `/api/admin/jobs`.

3.12 Audit Service

`lib/audit.ts` writes `audit_logs` (table, record, action, user, old and new values, changed fields, IP address, user agent, notes). `ensureAuditLogsTable` creates the table on demand; `getAuditContextFromRequest` extracts actor and network details. Writers are called after the business operation and must not make it fail. Reads are served by `actions/query/audit-logs.query.ts` and `/api/audit-logs` with filters by group, action, category, date, and text.

4. Data Design

4.1 Overview

All durable structured data is in one PostgreSQL 18 database, `infra_watch`, with the PostGIS extension. Files are in MinIO. The database has 44 base tables, 16 enforced foreign keys, and 73 secondary indexes. Schema is defined in TypeScript with Drizzle and changed by SQL migrations.

4.2 Database Architecture

4.2.1 Type and Connection

- Relational with `jsonb` for flexible payloads and PostGIS for spatial data.
- Driver `postgres` (postgres.js) through Drizzle (`drizzle-orm/postgres-js`); one pool per process, size `DATABASE_POOL_MAX` (default 10), idle timeout 20 s, maximum lifetime 1,800 s.
- Extensions installed: `plpgsql`, `postgis`. Embeddings are plain JSON, so `pgvector` is not required.
- Production refuses to start without `DATABASE_URL`.

4.2.2 Grouping by Domain

Domain	Tables
Identity	`user`, `session`, `account`, `verification`, `account_preferences`, `user_tour_progress`
Project mirror and reference	`projects`, `sync_logs`, `project_metric_snapshots`, `project_data_corrections`, `project_commodity_overrides`, `psgc_locations`, `regional_statistics`, `map_layers`
Engagement	`feedback`, `feedback_comments`, `feedback_comment_votes`, `issues`, `issue_responses`, `sms_grievance_reviews`, `contact_messages`, `contact_responses`, `saved_posts`, `post_subscriptions`, `submission_surveys`
Notifications	`notifications`, `notification_recipients`, `notification_reads`
Content	`articles`, `announcements`, `live_videos`, `email_templates`, `system_settings`
AI	`ai_chat_history`, `ai_chat_rate_limits`, `kb_documents`, `kb_chunks`, `mydas_dashboards`
Operations	`audit_logs`, `background_jobs`, `upload_sessions`, `analytics_events`, `analytics_daily_aggregates`
Platform	`spatial_ref_sys` (PostGIS)

4.3 Logical Data Model

Figures 6 and 7 show the entities and enforced relationships. The editable DBML and Mermaid files in `docs/erd/` contain every table and column and can be regenerated from the live database with `node docs/erd/generate-erd.cjs`.

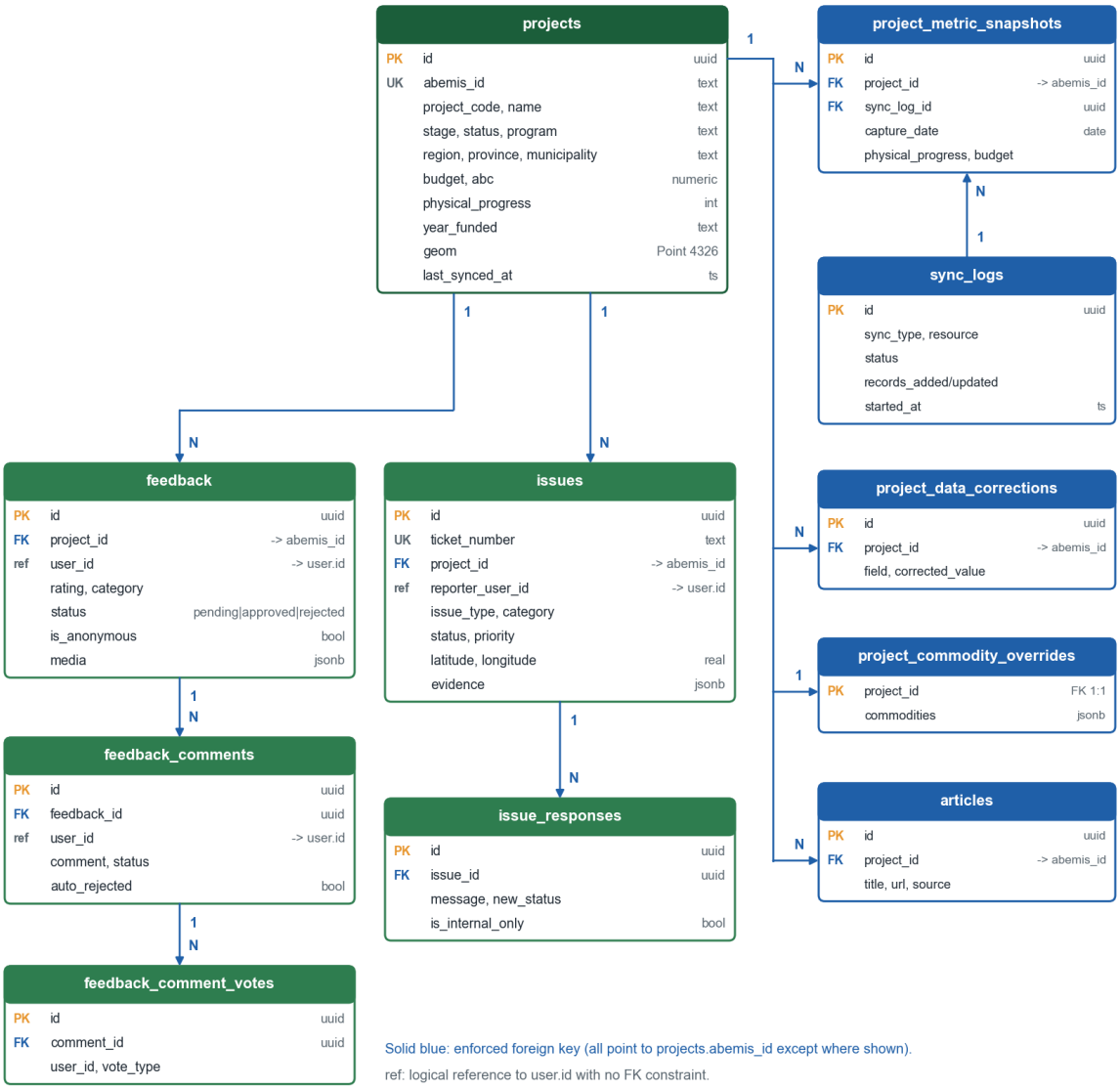


Figure 6. Logical model: projects, engagement, and synchronization.

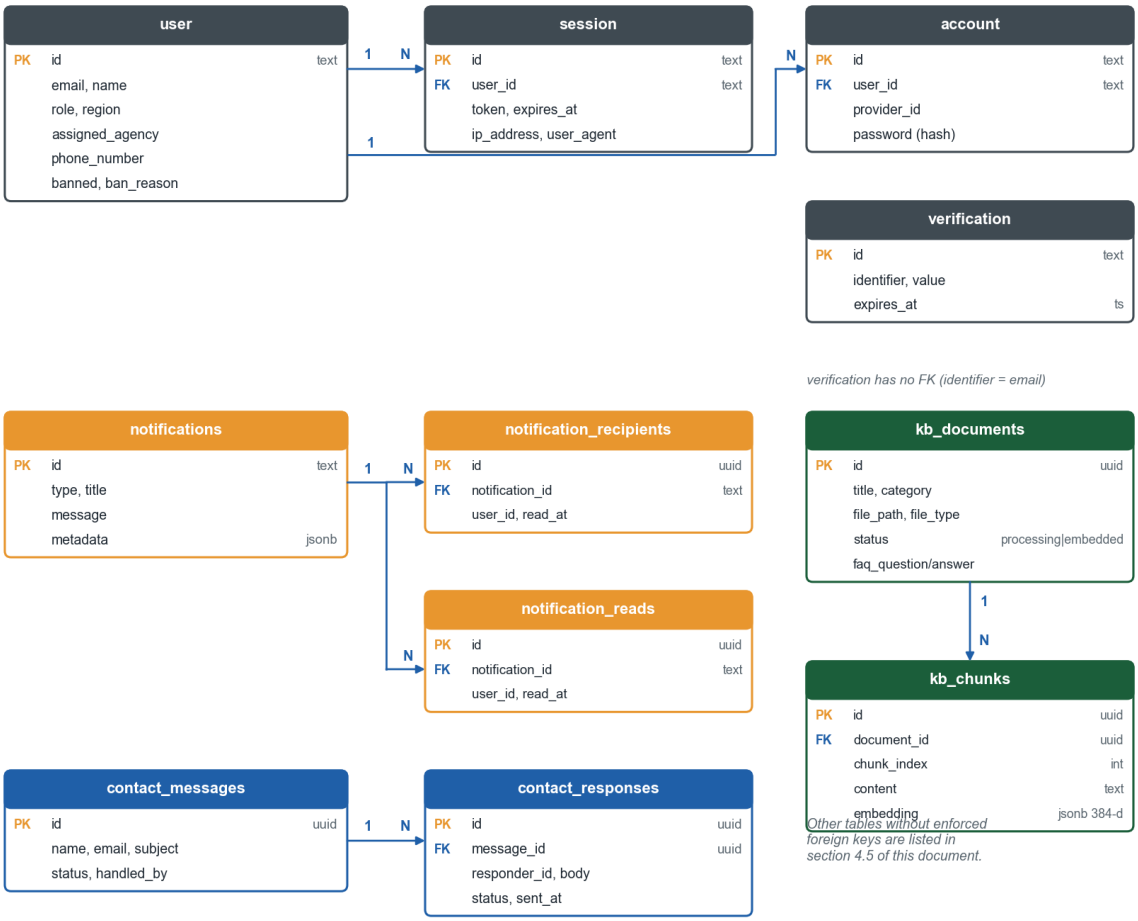


Figure 7. Logical model: identity, notifications, knowledge base, and contact.

4.4 Physical Data Model

4.4.1 Naming and Type Conventions

- Table and column names are `snake_case`; tables are plural (`projects`, `issues`) except Better Auth's own (`user`, `session`, `account`, `verification`).
- Primary keys are `uuid` with `gen_random_uuid()` for application tables and `text` for Better Auth tables.
- Timestamps are `timestamp` (or `timestamptz` for jobs); `created_at` defaults to `now()`; `updated_at` is set by the application.
- Money is `numeric(14,2)`; coordinates are `real` plus a `geometry(Point,4326)`; status values are plain `text` validated in code.
- Foreign keys to projects reference `projects.abemis_id`, the stable source key.

4.5 Key Entities and Attributes

4.5.1 Table `projects`

Column group	Columns	Notes
Keys	`id` (uuid, PK), `abemis_id` (unique), `abemis_raw_id` (unique), `project_code`	`abemis_id` is the join key for children
Description	`name`, `description`, `project_type`, `program`, `sub_program`, `banner_program`, `prexc_program`, `indicator_level1`, `indicator_level3`, `farm_operation`, `recipient_type`, `beneficiary`	`program` is `AMEFIP` or `INS`
Place	`region`, `province`, `municipality`, `barangay`, `district`, `psgc_code`, `latitude`, `longitude`, `geom`	`geom` is PostGIS Point 4326
Money	`budget`, `abc`, `contract_amount`, `budget_process`, `procurement_mode`	numeric(14,2)
Progress and time	`status`, `stage`, `physical_progress`, `financial_progress`, `calendar_days`, `start_date`, `target_completion_date`, `actual_completion_date`, `date_turn_over`	progress 0 to 100
Delivery detail	`contractor_name`, `implementing_agency`, `operating_unit`, `year_funded`, `quantity`, `quantity_unit`, `proposed_length`, `post_geotagged_length`, `road_*`, `implementation_type`	text
Raw and audit	`metadata` (jsonb), `commodities` (jsonb), `author`, `last_synced_at`, `created_at`, `updated_at`	`metadata` keeps source detail

Indexes: `projects_abemis_id_idx`, `projects_province_idx`, `projects_region_idx`, `projects_status_idx`, `projects_year_funded_idx`.

4.5.2 Tables `feedback`, `feedback_comments`, `feedback_comment_votes`

Column	Type	Description
`feedback.id`	uuid PK	Identifier
`project_id`	text FK	References `projects.abemis_id` (cascade)
`user_id`	text	Author (logical reference to `user.id`)
`rating`, `category`, `comment`	int, text, text	1 to 5 rating; general, quality, progress, concerns
`media`	jsonb	List of portal-hosted images and videos
`is_anonymous`	boolean	Hides the author on public pages only
`status`	text	`pending`, `approved`, `rejected`
`moderated_by`, `moderated_at`, `moderation_note`	text, timestamp, text	Decision record
`sla_*_at`, `auto_acknowledged_at`	timestamp	Reminder checkpoints
`sentiment`, `issue_type`, `helpful_count`	text, text, int	Classification and counters
`feedback_comments.*`	uuid PK; FK `feedback_id`	`comment`, `media`, `status`, `auto_rejected`, `rejection_reason`, vote counts, moderation fields
`feedback_comment_votes.*`	uuid PK; FK `comment_id`	`user_id`, `vote_type`

4.5.3 Tables `issues` and `issue_responses`

Column	Type	Description
`issues.id`, `ticket_number`	uuid PK, text unique	Identifier and public ticket
`project_id`	text FK	Optional link to `projects.abemis_id` (set null)
`reporter_user_id`, `reporter_name`, `reporter_contact`, `reporter_email`, `is_anonymous`	text, boolean	Reporter identity; staff only
`category`, `issue_type`, `reported_project_type`, `reported_farm_operation`	text	Classification
`status`, `priority`	text	`pending`, `reviewing`, `resolved`, `closed`; priority default `normal`
`description`, `public_description`, `public_approved_at`, `public_approved_by`	text, timestamp	Private text and approved public text
`region`, `province`, `municipality`, `barangay`, `landmark`, `latitude`, `longitude`	text, real	Location
`evidence`, `geo_video_track`, `geo_video_url`	jsonb, jsonb, text	Evidence list and geo-video track
`assigned_to`, `resolved_at`, `sla_*_at`	text, timestamp	Handling and SLA checkpoints
`issue_responses.*`	uuid PK; FK `issue_id` (cascade)	`responder_*`, `message`, `status_change`, `new_status`, `internal_notes`, `is_internal_only`, `attachment_urls`

4.5.4 Tables `user`, `session`, `background_jobs`, `audit_logs`

Table	Notable columns
`user`	`id` (text PK), `email` (unique), `email_verified`, `role` (default `citizen`), `region`, `assigned_agency`, `phone_number`, `banned`, `ban_reason`, `ban_expires`
`session`	`id`, `token`, `user_id` (FK, cascade), `expires_at`, `ip_address`, `user_agent`, `impersonated_by`
`background_jobs`	`kind`, `status` (default `queued`), `dedupe_key` (partial unique index while active), `payload`, `progress`, `result`, `attempts`, `max_attempts` (3), `available_at`, `lease_until`, `lease_token`, `error`
`audit_logs`	`table_name`, `record_id`, `action`, `user_id`, `user_name`, `old_values`, `new_values`, `changed_fields`, `ip_address`, `user_agent`, `notes`, `created_at`

4.5.5 Knowledge, Notification, and Other Tables

Table	Notable columns
`kb_documents`	`title`, `category`, `file_type`, `file_name`, `file_path`, `file_size`, `chunk_count`, `status`, `faq_question`, `faq_answer`, `content_preview`, `error_message`, `uploaded_by`
`kb_chunks`	`document_id` (FK, cascade), `chunk_index`, `content`, `embedding` (jsonb, 384 numbers), `token_count`
`notifications`, `notification_recipients`	`type`, `title`, `message`, `metadata`; recipient `user_id`, `read_at`

Table	Notable columns
`sync_logs`	`sync_type`, `resource`, `status`, `records_added`, `records_updated`, `records_failed`, `total_processed`, `errors`, `error_details`, `started_at`, `completed_at`, `duration`, `triggered_by`
`project_metric_snapshots`	`project_id`, `sync_log_id`, `capture_date`, `physical_progress`, `financial_progress`, `budget`, `abc`, `status`, `target_completion_date`, `program`
`ai_chat_history`	`conversation_id`, `user_id`, `user_message`, `assistant_message`, `status`, `provider`, `model`, `tool_names`, token counts, `duration_ms`, `finish_reason`

4.6 ORM and API Integration

Queries are written with Drizzle's query builder. The schema modules export typed tables, and query functions compose conditions, including the scope conditions from Section 3.2.5.

```
// lib/db/index.ts (excerpt)
const sql = postgres(databaseUrl, {
  max: Number(process.env.DATABASE_POOL_MAX ?? 10),
  idle_timeout: Number(process.env.DATABASE_IDLE_TIMEOUT ?? 20),
  max_lifetime: Number(process.env.DATABASE_MAX_LIFETIME ?? 1800),
});
export const db = drizzle(sql, { schema: { ...schema, ...authSchema } });

// typical scoped query
const rows = await db.select().from(projects)
  .where(and(...getProjectScopeConditions(user), ilike(projects.name, `%${q}%`)))
  .orderBy(sql`last_synced_at desc`).limit(pageSize).offset(offset);
```

4.7 Data Integrity and Validation

Layer	Mechanism
Database	Primary and unique keys; 16 foreign keys with explicit delete rules; partial unique index for active jobs; not-null defaults on timestamps
Application	Zod schemas at every boundary; typed Drizzle inserts; status constants; normalization helpers
Files	Path pattern checks (`feedback/<ts>-<hex>.<ext>`), MIME and signature checks, folder allowlists
Concurrency	Row version in `sms_grievance_reviews`; job leases with fencing token; transactions for notification persistence, live-video state, and knowledge reindex swaps

4.8 Indexing, Caching, and Optimization

- Indexes support the filters and joins used by pages: projects (id, province, region, status, year), issues (status, project, ticket), feedback (status, project, sentiment), `kb_chunks` (document and chunk), `background_jobs` (claim and dedupe), analytics tables (date and metric).
- Public reads use `cachedPublic`; admin-visible cache clear is audited.
- Analytics use bounded row counts and fetch only needed columns; snapshots avoid recomputing history.
- Spatial queries use the PostGIS point column; the browser clusters markers.

4.9 Migrations, Backup, Archiving, and Retention

- **Migrations:** 39 SQL files in `drizzle/` with a journal. At container start `scripts/prepare-migrations.ts` prepares the journal and `scripts/migrate.mjs` applies pending files one statement at a time, treating "already exists" as a no-op; it replaces `drizzle-kit migrate`, which could roll back an entire run on a Postgres notice under Bun. `RUN_DB_MIGRATIONS=false` skips this step. `BASELINE_EXISTING_DATABASE=true` is a one-time opt-in for a database that predates the journal.
- **Rollback:** reverse SQL for selected migrations is kept in `docs/migrations/`.
- **Backup:** database and bucket backups are an operational task outside the application; schedule them and test restores.
- **Retention:** see RAD Section 4.7 (chat history up to 365 days, snapshots 365 days, analytics events pruned, audit log kept until an administrator removes entries).

5. Interface Design

5.1 Interface Principles

- **Server actions first.** Pages and forms use server actions (`actions/query`, `actions/mutation`). Route handlers exist where a browser, an external system, a stream, or a file transfer needs an HTTP endpoint.
- **JSON over HTTPS.** Bodies are `application/json` unless a route accepts multipart uploads. Staff JSON bodies are limited to 32 KB by default and must declare the content type.
- **Session cookies.** Authentication is a Better Auth session cookie; there are no API keys for users.
- **Origin checks.** Mutating staff endpoints reject a mismatching `Origin` header.
- **No version prefix.** Routes are unversioned (`/api/...`) because the only API client is this application; breaking changes are made together with the pages that call them.
- **Public-safe by construction.** Public routes return purpose-built objects, never raw rows.

5.2 Error and Success Response Format

Two response conventions exist, matching the two kinds of endpoint:

Endpoint family	Success	Error
Staff and operational routes (`/api/admin/*`, `/api/health`, jobs, sync)	Plain JSON object or list; HTTP 200, 201, or 202	`{ "error": "message" }` with 400, 401, 403, 404, 413, 415, or 5xx; produced by `apiError`
Participation routes (feedback, issues, uploads)	`{ "success": true, "data": {...}, "message": "..." }`, HTTP 201 on create	`{ "success": false, "error": "message" }` with the matching status
Public analytics (`/api/public/[view]`)	`{ "data": ..., "dataAsOf": "<ISO>", "filters": {...} }`	`{ "error": "..." }`, 404 for unknown view, 503 when project data is unavailable

`apiError` maps Zod validation errors to 400 with the issue messages joined, uses the error's `status` for known conditions, and replaces any 5xx detail with "Service unavailable. Try again." so internals are never exposed. Server errors are logged by name only.

5.3 Authentication Endpoints

Method	Route	Access	Purpose
GET, POST	`/api/auth/[...all]`	Public (rate and origin rules by Better Auth)	Sign-up, sign-in, sign-out, email OTP, password reset, OAuth callbacks, session lookup, admin plugin operations

5.4 Public Endpoints

Method	Route	Purpose
GET	`/api/projects`, `/api/projects/[id]`	Project list and detail
GET	`/api/public/projects`, `/api/public/projects/[id]`	Public-safe project list and detail
GET	`/api/public/[view]`	Analytics views over a cached project snapshot (`places` search and named views); `Cache-Control: public, max-age=60, stale-while-revalidate=300`
GET	`/api/public/project-export`	Filtered project export; `format=csv` returns `infrawatch-projects.csv`
GET	`/api/public/regional-statistics`	Regional statistics for a year and program, cached 60 s
GET	`/api/public/articles`, `/api/public/site-notices`, `/api/announcements/active`	Articles, maintenance and banner notices, active announcements
GET	`/api/map-layers`, `/api/kml-proxy`	Overlay definitions; bounded KML fetch
GET	`/api/evidence`	Public-safe geolocated evidence
GET, POST	`/api/issues`	Public E-Report list; create an E-Report
GET	`/api/issues/[id]`, `/api/issues/[id]/public-evidence/[index]`	Public-safe or owner-aware detail; approved evidence image
GET	`/api/projects/[id]/feedback`	Approved feedback for a project
POST	`/api/analytics/events`	Privacy-minimal usage event, rate limited
POST	`/api/chat`	Public assistant (streamed)

5.5 Signed-In Citizen Endpoints

Method	Route	Purpose
POST	`/api/projects/[id]/feedback`	Submit feedback
PATCH, DELETE	`/api/projects/[id]/feedback/[feedbackId]`	Owner edits or deletes own feedback
GET	`/api/my-feedbacks`, `/api/my-issues`	Owner lists
GET, POST	`/api/notifications`	List notifications; mark read
GET	`/api/notifications/stream`	Server-sent events for live notifications
POST	`/api/upload`	Validated upload
GET, POST, PUT, DELETE	`/api/upload/multipart`	Resumable multipart upload for large video
GET	`/api/upload/preview`	Authorized preview of stored media
POST	`/api/submission-survey`; GET `/api/submission-survey/status`	Optional survey and gate status

Example: submit feedback.

```
POST /api/projects/{id-or-code}/feedback
{ "comment": "The canal lining is cracked near the pump house.",
  "category": "concerns", "issueType": "irrigation_leak", "rating": 2,
  "isAnonymous": true,
  "media": [ { "type": "image", "url": "feedback/1760000000000-<32 hex>.jpg" } ] }

201 { "success": true,
      "data": { "id": "...", "status": "approved", "unhelpfulCount": 0, "commentCount": 0 },
      "message": "Feedback submitted and approved automatically. It's now visible on the project
page." }
400 { "success": false, "error": "Comment must be 1000 characters or less" }
401 { "success": false, "error": "Authentication required" }
```

Example: submit an E-Report (form data). Fields: `projectId` (optional), `category`, `description`, `isAnonymous`, `reporterName`, `reporterContact`, `reporterEmail`, `region`, `province`, `municipality`, `barangay`, `landmark`, and `evidence` files. The JSON path accepts a canonical `evidence` array with coordinates and a geo-video track instead of files. On success the response is HTTP 201 with the created record and the message "Issue report submitted. Tracking ticket INFRA-<year>-<six digits> has been created."

5.6 Staff Endpoints

Method	Route	Access	Purpose
GET	`/api/admin/analytics`, `/breakdown`, `/drillthrough`, `/progress-curve`, `/cost-by-region`	Staff, analytics view	Managerial dashboard data (scoped)
POST	`/api/admin/analytics/assistant`	Staff, managerial AI enabled	Copilot and executive brief
GET	`/api/admin/citizen-engagement`	Staff	Engagement analytics
GET	`/api/admin/projects`, `/api/admin/projects/stats`	Staff (scoped)	Admin project list and statistics
DELETE, PATCH	`/api/admin/projects/[id]/commodities`	Staff	Commodity override
GET	`/api/admin/feedback`, `/stats`	Staff (scoped)	Feedback queue and counts
PATCH	`/api/admin/feedback/[feedbackId]/moderate`	Staff (scoped)	Approve or reject
DELETE	`/api/admin/feedback/[feedbackId]`	Staff (scoped)	Delete feedback
GET, PATCH	`/api/admin/comments`	Staff	Comment moderation list and decision
GET	`/api/admin/issues`, `/stats`	Staff (scoped)	E-Report queue and counts
GET, DELETE	`/api/admin/issues/[id]`	Staff (scoped)	Detail; delete
POST	`/api/admin/issues/[id]/responses`	Staff (scoped)	Response and status change
PATCH	`/api/admin/issues/[id]/evidence/[index]`	Staff (scoped)	Approve or withdraw one image for public display
GET	`/api/admin/sms-grievance/attention`	Staff	Count of SMS items needing attention
GET	`/api/admin/data-quality`, `/export`	Staff	Findings and export
PATCH	`/api/admin/data-quality/projects/[projectId]`	Staff	Always returns 405 (recommendation-only)

Method	Route	Access	Purpose
GET, POST	`/api/admin/sync`	Staff view; admin trigger	Sync statistics; start a sync
GET	`/api/admin/sync-logs`	Staff	Sync history
GET, POST, PATCH	`/api/admin/jobs`; GET `/api/admin/jobs/[id]/stream`	Admin	List, enqueue, retry; job progress stream
GET	`/api/admin/locations`	Staff	Location options
GET, PUT, DELETE	`/api/admin/regional-statistics`	Staff	Regional statistics maintenance
GET	`/api/admin/chat-history`	Staff (role-based visibility)	Assistant history
GET	`/api/audit-logs`	Staff, audit view	Audit entries
GET, DELETE	`/api/health`	Admin	Service health; clear public cache
POST	`/api/knowledge-base/upload`, `/process`; GET `/files/[id]`	Staff, knowledge-base permissions	Upload, index, download
POST	`/api/voice/wake-token`, `/api/voice/transcribe`	Admin	Voice assistant support

5.7 Server Actions

Each file groups the actions of one domain. Queries are read-only and cached per request where useful; mutations validate input, check permission and scope, write audit, and call `revalidatePath` for the pages affected.

Group	Files
Queries	`public-projects`, `project-preview`, `analytics`, `activity-feed`, `community-stats`, `feedback`, `feedback-comments`, `feedback-votes`, `issues`, `reports`, `audit-logs`, `users`, `settings`, `knowledge-base`, `live-videos`, `contact-messages`, `get-location-options`, `mydas-dashboard` (each `*.query.ts`)
Mutations	`auth`, `feedback`, `feedback-comment`, `feedback-vote`, `post-interactions`, `profile`, `contact`, `suggestion`, `content`, `email-templates`, `knowledge-base`, `live-videos`, `settings`, `sms-grievance`, `users`, `mydas`, `mydas-dashboard` (each `*.mutation.ts`)

5.8 External Interface Contracts

5.8.1 ABEMIS

```
GET {ABEMIS_BASE_URL}/api/infra-amefip-list?page=1&page_size=500[&year_funded=2025]
x-api-key: <ABEMIS_API_KEY>

200 { "data": [ { ...project... } ], "pagination": { "total_pages": N, ... } }
```

The client retries up to three times with a delay that grows each time, refuses redirects, and refuses a target on a different origin from `ABEMIS_BASE_URL`. The endpoint path can be overridden with `ABEMIS_INFRA_ENDPOINT` but must be root-relative.

5.8.2 SMS Gateway and Feed

Outbound: `POST {SMS_API_URL}` with a JSON body containing the sender identifier, account username and password from the environment, the recipient normalized to `+639XXXXXXXXX`, and the message; the function returns `{ success, error?, data?, status? }` and never throws. Inbound: `GET {SMS_GRIEVANCE_API_URL}` returns `{ messages: [...] }` where each message has an identifier, SMS identifier, sender number, content, receive date and time, location, status, month, and year; records failing the shape check are ignored.

5.8.3 Email

`sendEmail({ to, subject, text, html })` uses Nodemailer with `MAIL_HOST`, `MAIL_PORT` (default 587), credentials, and `MAIL_ENCRYPTION`; on port 587 it requires STARTTLS and refuses to send in clear text. It returns `{ success, messageId?, error? }`; missing configuration returns an error rather than throwing.

5.9 Real-Time Interfaces

Stream	Route	Content
Notifications	`GET /api/notifications/stream`	`text/event-stream`; one JSON notification per event for the signed-in user (type, title, message, metadata, created time)
Job progress	`GET /api/admin/jobs/[id]/stream`	Progress object (`processed`, `total`, `message`) until the job ends

5.10 Rate Limiting and Request Bounds

Endpoint	Limit
`POST /api/chat`	64 KB body; per-identity per minute (default 5); global per day (default 2,000); response timeout `CHAT_RESPONSE_TIMEOUT_MS`; client address from nginx-supplied headers only when `CHAT_TRUST_PROXY=true`
`POST /api/analytics/events`	Per-client rate limit; bounded event size
Staff JSON routes	32 KB body, `application/json` required
Uploads	Type, signature, and size limits; 110 MB at nginx; 50 MB parts for multipart
MYDAS	Rate-limited mutation; 5-second statement timeout; at most 1,000 rows

5.11 Pagination, Filtering, and Sorting

- Public directory: 20 per page; `page` and filter parameters are parsed and bounded by `lib/public-project-directory.ts`.
- Admin lists: `page` and `pageSize` clamped to 10 to 100 (default 25).
- Sorting is a closed set per list (for example latest sync, name, budget, funding year); unknown values fall back to the default.
- Search is case-insensitive substring matching on a fixed set of columns per list.

5.12 Caching Headers

Responses that depend on the user, such as the service health check, use `Cache-Control: private, no-store`; public analytics views use `public, max-age=60, stale-while-revalidate=300`. The application sets `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: credentialless` on every response.

6. Workflow Design

6.1 Overview

INFRA Watch has no configurable workflow engine. Its workflows are fixed state machines implemented in code: feedback moderation, E-Report handling, SMS grievance review, document indexing, account state, and live-video scheduling. This chapter records their states, who may move them, and what each move triggers.

6.2 State Machines

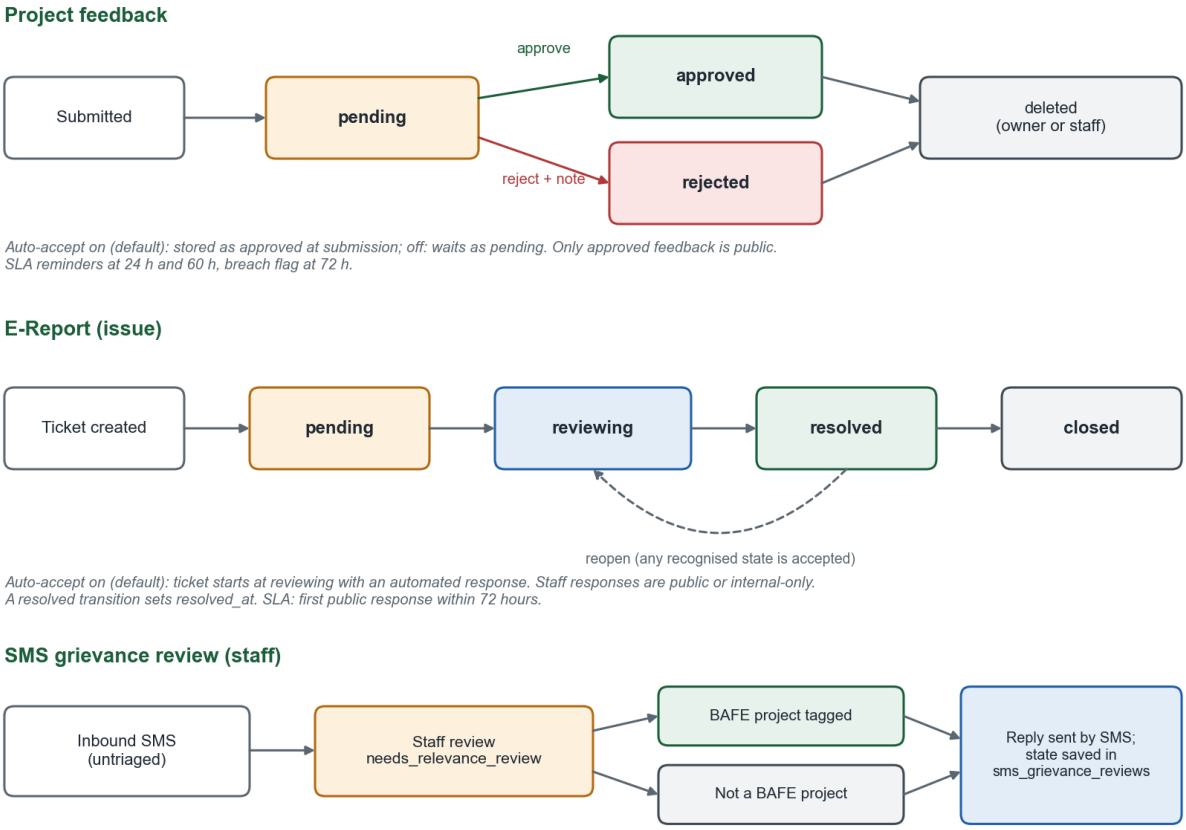


Figure 8. Feedback, E-Report, and SMS grievance states.

6.3 Feedback Workflow

From	To	Actor	Effect
(new)	`approved`	System, when auto-accept is on (default)	Citizen acknowledged; item public after automated checks
(new)	`pending`	System, when auto-accept is off	Waits for a moderator; SLA clock runs
`pending`	`approved`	Moderator in scope, regional admin, admin	Public; moderator and time stored; citizen notified

From	To	Actor	Effect
`pending` or `approved`	`rejected`	Same	Hidden from public; note shown to owner
any	deleted	Owner or staff	Row and attachments removed; audited

Comments carry their own `status` and an `auto_rejected` flag set by the profanity pre-check, and are reviewed on `/comment-moderation`.

6.4 E-Report Workflow

From	To	Actor	Effect
(new)	`pending` or `reviewing`	System (auto-accept setting)	Ticket created; staff notified; automated first response when auto-accept is on
`pending`	`reviewing`	Staff in scope	Handling started
`reviewing`	`resolved`	Staff in scope	`resolved_at` set; reporter notified
`resolved`	`closed`	Staff in scope	Final
any recognised state	any recognised state	Staff in scope	Allowed by the server; no enforced transition graph

Each response is public or internal-only, may carry attachments, and may change status and approve the public summary. Evidence images are approved individually for public display.

6.5 SLA Monitoring and Escalation

1. Every 15 minutes the job selects open issues and pending feedback by age.
2. At 24 hours: gentle reminder to the scoped moderators (checkpoint `sla_gentle_reminder_at`).
3. At 60 hours: urgent reminder to moderators and administrators.
4. After 72 hours: breach notice to administrators; the record is flagged in SLA reports.
5. A real response (first non-internal issue response, or a moderation decision on feedback) ends the clock.

6.6 SMS Grievance Workflow

1. Inbound message appears in the review list as untriaged.
2. If the sender has an open case within a week, the auto-link job folds the message into it.
3. If enabled, the auto-acknowledge job texts a first reply asking for project type, name, age, gender, and location (once per message).
4. Staff tag an actual BAFE project, mark Not a BAFE project, or link a follow-up. The state is saved with a version check, then the SMS is sent.
5. Replies are further SMS entries in the thread; internal notes stay internal.

6.7 Notification Workflow

1. A domain event (new feedback, new E-Report, response, SLA checkpoint, ban, and so on) calls the notification service with a type, text, metadata, and the candidate recipients.

2. The service filters recipients by scope and each user's channel preferences.
3. One `notifications` row and the `notification_recipients` rows are written in a transaction.
4. Connected users receive the event on their stream; `notification-delivery` jobs send email and SMS.
5. The user opens the item; the read state is stored.

6.8 Knowledge Document Workflow

State	Meaning	Moves to
`processing` / `indexing`	Uploaded or entered; text being extracted, chunked, embedded	`embedded` or `failed`
`embedded`	Searchable by the assistant	`archived`; reindex keeps the old chunks until the new ones are ready
`failed`	Extraction or embedding failed; a sanitized reason is stored	Reindex
`archived`	Excluded from retrieval	Restored to active, or permanently deleted

6.9 Account and Live-Video States

- **Account:** unverified, verified (active), banned (optionally until `ban_expires`), deleted. Administrators move accounts between states with self-protection rules.
- **Live video:** inactive, scheduled (published in the future), live, featured, expired. A transaction guarantees at most one live and one featured entry at a time.

6.10 Design Constraints

- Staff transitions are limited by permission and scope, not by a transition graph (see limitation R-07).
- Auto-accept is a global setting; there is no per-region override.
- Workflow side effects (audit, notification) are separate writes and are not wrapped in one transaction with the state change.

7. Security Design

7.1 Overview

INFRA Watch handles citizen contact details, geotagged photographs, and staff decisions, and it publishes content under a government name. The design therefore focuses on controlling who can see and change what, keeping private evidence private, limiting what automated components can do, and recording what staff did.

7.2 Security Objectives

- Only authorized people can read private data or change records.
- Moderators see only the region and program they are assigned.
- Public output never exposes contact data, internal notes, or private evidence.
- Uploaded files cannot harm the system or other users.
- AI features cannot be steered to reveal internals or run harmful queries.
- Every staff action and security event is attributable.

7.3 Security Architecture Overview

Layer	Controls
Network and proxy	TLS (Let's Encrypt) at nginx; HTTP redirected; app bound to loopback; nginx replaces client-supplied `X-Forwarded-For`; chat endpoint unbuffered with a 65-second timeout; 110 MB body limit
Application edge	Trusted-origin list; origin check on staff mutations; JSON content-type and size checks; COOP and COEP headers
Identity	Better Auth sessions, email OTP, optional Google, bans, session revocation
Authorization	Role statements, moderator region and program scope, fail-closed defaults, admin-only operations
Data	Public DTOs, private bucket, hashed passwords, minimal analytics, retention limits
Content	Upload validation, NSFW checks, profanity screening, per-image public approval
AI	Bounded read-only tools, refusal policy, SQL guardrails, rate limits, history expiry
Monitoring	Audit log, sync logs, health check, failed-login and blocked-upload events

7.4 Authentication Design

7.4.1 Mechanism

Email and password (with required verification), email one-time codes, and optional Google sign-in. Passwords are hashed by Better Auth. Codes are 6 digits and expire after `EMAIL_OTP_EXPIRY` seconds (default 180). The OTP plugin cannot create accounts.

7.4.2 Session Management

Sessions are rows in `session` with expiry, IP address, and user agent and are carried in a cookie issued by Better Auth's Next.js plugin. Banning a user blocks new sessions; an administrator can end all sessions of a user. `getSession` is cached per request so repeated guard calls do not repeat database reads.

7.5 Authorization and Scope

7.5.1 Role, Resource, and Action Model

Statements in `lib/permissions.ts` list, for each resource, the allowed actions per role (see RAD Section 2.6 for the matrix). `requirePermission` throws a 403 error when the role lacks the action. The Better Auth admin plugin uses the same statements for its built-in operations.

7.5.2 Scope Resolution

1. Determine the caller's role.
2. If `moderator`: require at least one of region or program; if neither, deny everything (`false` condition).
3. Add `projects.region ILIKE region` and `projects.program = program` as assigned.
4. Always add the fiscal-year condition.
5. For a single record (feedback, issue), call `checkModeratorScope`/`checkIssueScope`, which resolves the record's project and applies the same conditions; deny on mismatch.

7.6 Data Protection

- **Public objects:** `public-issue-dto.ts` and project/feedback DTOs include only fields marked public; media and geotags are sanitized.
- **Private objects:** issue evidence and knowledge-base files live in the private bucket and are streamed by authenticated handlers that re-check ownership or scope on every read.
- **Anonymity:** feedback marked anonymous is displayed without the author but keeps `user_id` for moderators; E-Reports marked anonymous store no name or contact data.
- **Retention:** short retention for chat history and snapshots; analytics events pruned; deletion pages describe the citizen's options.
- **Secrets:** database, auth, SMS, SMTP, storage, and AI credentials are environment variables only.
- **Known gap:** stored images keep EXIF metadata (R-05); recommended fix is metadata stripping for public media.

7.7 Upload Security

1. Require a session; check the destination folder against an allowlist.
2. Check declared MIME type, file extension, magic bytes, and size.
3. Run NSFWJS on images and on sampled video frames (fail-closed available).
4. Write only to an application-generated name (`<timestamp>-<32 hex>.<ext>`); never use the client file name as a path.
5. Record blocked and NSFW-rejected uploads in `audit_logs`.
6. Serve public media only from approved prefixes; serve private media through authorization checks.

7.8 Security for Inter-Service and External Interfaces

- ABEMIS: API key from the environment; fixed origin; redirects refused; retries bounded.
- KML proxy: size limit and URL checks to avoid server-side request abuse.
- SMS and SMTP: credentials server-side; failures returned as values without leaking provider detail.
- AI providers: only bounded, necessary context is sent; no raw user identity.
- Storage start-up check: refuse to run if the private bucket would allow anonymous reads.

7.9 AI Safety Design

Risk	Control
Prompt injection from retrieved data	Retrieved fields are labelled untrusted; tools return data only; injection phrases refused
Revealing internals or users	Policy refuses such requests; the model never sees raw identity; aggregate user counts only on the admin surface
Harmful generated SQL (MYDAS)	Single `SELECT`, keyword and table allowlists, row limit, 5-second timeout inside a transaction, admin only. Column access is controlled by the prompt, not by the validator
Cost and abuse	Request and history size limits, per-minute and daily limits, optional sign-in requirement
Wrong answers	Grounding in tools, verification notice on generated analysis, advisory wording

7.10 Security Monitoring and Auditing

The audit log records record changes, failed sign-ins, blocked and NSFW-rejected uploads, sync runs, and cache clears, with the actor, IP address, and user agent. Staff review it in `/audit-logs`. Sync history and job progress show operational anomalies. The service health page shows database, object storage, and Redis status.

7.11 Privacy and Compliance (RA 10173)

Principle	Design response
Transparency	Privacy notice, terms of service, and data deletion pages
Legitimate purpose and proportionality	Only fields needed for handling a report are collected; survey is optional
Security	Controls in this chapter
Data subject rights	Staff can delete a submission or an account; owners can edit and delete their own feedback
Retention	Limits in Section 4.9 and RAD Section 4.7

7.12 Security Headers and Browser Protections

- `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: credentialless` on all responses.

- `Secure` and `SameSite` cookie attributes in production through Better Auth.
- Strict image sources in `next.config.ts` (own origin, the object-storage host, Google avatar host, ABEMIS host, and local MinIO in development).
- Recommended addition: a `Content-Security-Policy` and `Strict-Transport-Security` header at nginx; neither is set in the sample configuration.

7.13 Summary of Controls and Known Gaps

Area	Status	Note
Role and scope enforcement	Implemented	Fail-closed for moderators
Private evidence	Implemented	Bucket policy reconciliation at start-up
Upload validation and NSFW check	Implemented	Fail-closed optional
Audit log	Implemented	No automatic purge
Direct plugin administration limit	Confirm	Review finding S1
Dependency advisories	Confirm	Review finding S2; audit in CI
EXIF stripping	Not implemented	R-05
Knowledge-base visibility state	Not implemented	R-06
CSP and HSTS headers	Not configured	Recommend at nginx

8. Error Handling and Logging

8.1 Objectives

- Tell users what they can fix and nothing more.
- Never expose stack traces, SQL, file paths, or credentials.
- Keep enough information in logs and the audit table to diagnose and to account for actions.
- Make background failures visible and retryable.

8.2 Error Taxonomy

Class	HTTP	Examples	Handling
Validation	400	Missing comment; coordinates out of range; unsupported media	Message returned; no write
Authentication	401	No session	Redirect to sign-in or JSON error
Authorization	403	Citizen on staff route; out-of-scope record; cross-origin mutation	Denied; sensitive denials audited where relevant
Not found	404	Unknown project, issue, or view	Generic message
Payload	413, 415	Body too large; wrong content type	Rejected before parsing
Method	405	Data-quality project `PATCH`	Deliberate, with explanation
Dependency	503	Project data temporarily unavailable (public analytics)	Generic message; logged by name
Internal	500	Unexpected exception	"Service unavailable. Try again."; logged

8.3 Client Error Messages

Staff APIs return `{ "error": "..." }`. Participation APIs return `{ "success": false, "error": "..." }`. Upload errors are mapped to friendly text by `getClientUploadErrorMessage`. AI endpoints return a single generic error message for failures. Interfaces show unavailable, unknown, stale, and cannot-be-assessed states in plain words rather than blank values.

8.4 Logging Design

Log	Content	Where
Application log	Operational events and server error names; scheduler and sync progress; no secrets or personal data	Container stdout (Docker logs), plus `.dev-logs` in development
Audit log	Business and security events with actor, old and new values, IP address, user agent	`audit_logs` table; `/audit-logs`
Sync log	Per-run counts, errors, duration, trigger	`sync_logs`; `/sync`
Job state	Status, attempts, progress, result, error	`background_jobs`; `/api/admin/jobs`

Log	Content	Where
Chat history	Turn text, provider, model, tools used, tokens, duration, status	`ai_chat_history` (expiring)
Web analytics	Page usage when consented	Google Analytics 4 (optional)

8.5 Audit Entries

An audit row has: `table_name`, `record_id`, `action` (for example CREATE, UPDATE, DELETE, or a named event), `user_id`, `user_name`, `old_values`, `new_values`, `changed_fields`, `ip_address`, `user_agent`, `notes`, `created_at`. Staff mutations call `logAudit` with a context built by `getAuditContextFromRequest` so IP and user agent are included.

8.6 Failure Handling by Component

Component	On failure	Recovery
ABEMIS sync	Run marked failed or completed with warnings; counts and error details stored	Next scheduled tick; manual run; retry from `/api/admin/jobs`
Background job	Attempts incremented; retried with back-off up to the maximum; then `failed`	Admin retry (`PATCH /api/admin/jobs`)
SMS send	Decision already saved; thread entry marked failed	Staff resend from the Reply tab
Email	Returns an error value; delivery job retried	Fix SMTP settings; retry
MinIO	Upload error to the user (or local fallback in code, see R-04)	Restore storage; reconcile on restart
Redis	Ignored; memory cache continues; retry after 30 seconds	None needed
AI provider	Generic error; history marked failed	Retry; switch provider by configuration
Database	Requests fail; container health depends on it	Restore database; start-up waits up to 30 attempts

8.7 Monitoring and Alerting Recommendations

- Alert when no successful sync has completed in 24 hours.
- Alert on a growing number of failed jobs or repeated failed sign-ins.
- Watch MinIO and PostgreSQL disk and backup status.
- Review SLA breach counts weekly.
- Collect container logs centrally and keep them separate from the audit table, which is for accountability rather than diagnostics.

8.8 Example Scenarios

Scenario A: ABEMIS returns HTTP 526 at the scheduled hour

The client retries three times and the run is logged as failed with the error text. No project data changes. Three hours later the schedule tries again; the last-sync time on public pages shows the data's true age until a run succeeds.

Scenario B: a citizen uploads a renamed executable as a photo

The signature check does not match the declared image type, so the upload is rejected with a validation error, nothing is stored, and an audit entry records the blocked file name, type, and size.

Scenario C: two reviewers reply to the same SMS

Both load version 3 of the review. The first save becomes version 4 and the SMS is sent. The second save sees a version mismatch and is rejected, so the sender receives one reply.

9. Deployment Design

9.1 Topology

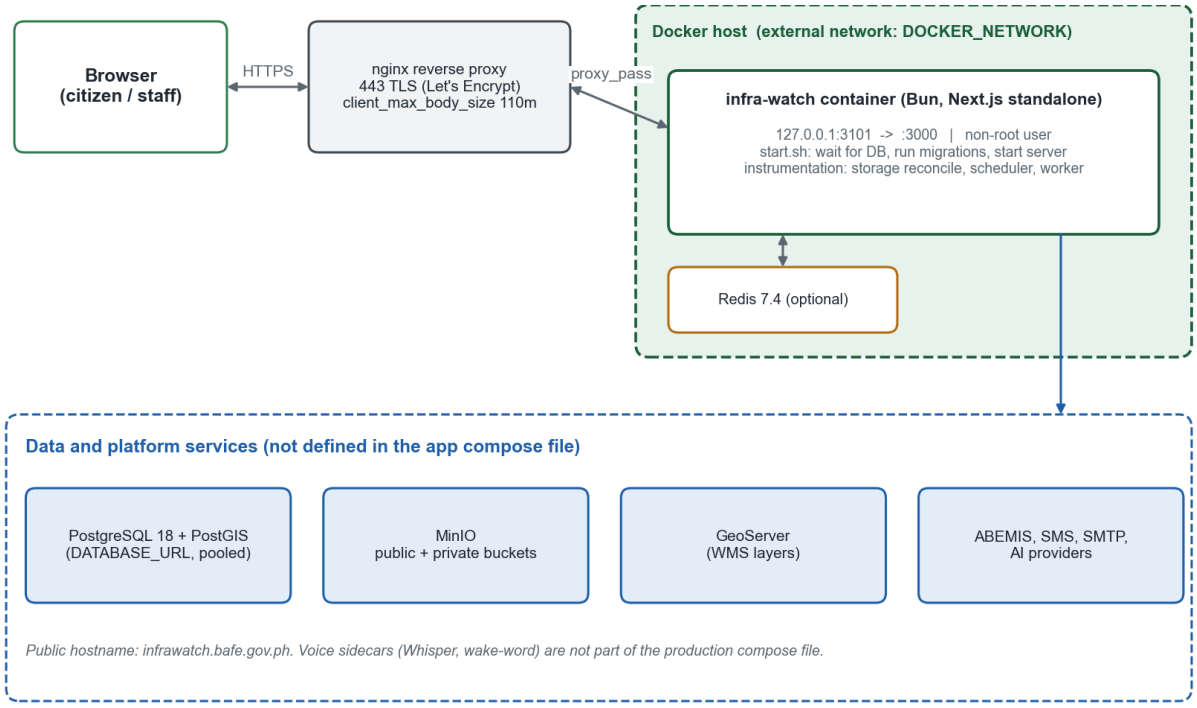


Figure 9. Deployment topology.

9.2 Build and Container

Stage	What it does
`dependencies`	`bun install --frozen-lockfile` on `oven/bun:1.3.14-slim`; preloads the GeoIP database
`builder`	Copies sources; passes public build arguments (`NEXT_PUBLIC_APP_URL`, `NEXT_PUBLIC_MINIO_*`, `NEXT_PUBLIC_GEOSERVER_*`, `NEXT_PUBLIC_GA_MEASUREMENT_ID`); runs `bun run build` with placeholder database and auth values
`runner`	Creates a non-root user; installs ffmpeg; copies the standalone output, static files, `lib`, migrations, scripts, and `docker/start.sh`; the container command `/app/start.sh` waits for PostgreSQL, applies migrations, then runs `bun server.js`; the image has a 30-second HTTP health check on port 3000

Public build arguments are inlined into the client bundle, so changing the public URL, bucket host, GeoServer URL, or analytics ID requires a rebuild.

9.3 Compose and Reverse Proxy

- `docker-compose.production.yml` defines the `app` service (restart `unless-stopped`, `init: true`, port `127.0.0.1:${APP_PORT:-3101}:3000`, environment from `.env.production`) and an optional `cache`

service (Redis 7.4, append-only, 256 MB, LRU eviction) behind the `cache` profile. It joins an external Docker network (`DOCKER_NETWORK`).

- `deploy/nginx/infrawatch.bafe.gov.ph.conf` redirects HTTP to HTTPS, serves the ACME challenge, terminates TLS, sets `client_max_body_size 110m`, proxies `/api/chat` with buffering off and a 65-second timeout, and proxies everything else with upgrade support and 300-second timeouts. It replaces, rather than appends to, client-supplied forwarding headers.

9.4 Start-Up Sequence

1. `/app/start.sh` (from `docker/start.sh`) waits for PostgreSQL (up to 30 attempts, 2 seconds apart).
2. If `RUN_DB_MIGRATIONS` is not `false`: `scripts/prepare-migrations.ts`, then `scripts/migrate.mjs`.
3. `bun server.js` starts the standalone Next.js server as the non-root user.
4. `instrumentation.ts` (Node runtime, production): `ensureStorageBuckets` reconciles public and private bucket policies and aborts on a collision or unsafe policy; then the scheduler starts, then the background worker.

9.5 Configuration

All configuration is by environment variables (Appendix A). Production requires `DATABASE_URL`, `BETTER_AUTH_SECRET`, and `BETTER_AUTH_URL`; optional groups (SMS, AI, voice, Redis, analytics) switch features on or off. Secrets belong in `.env.production` on the host or a secret store and never in the repository.

9.6 Release Checklist

Step	Command or action
Type check	`bunx tsc --noEmit`
Lint	`bun run lint --quiet`
Tests	`bun test --timeout 30000`
Build	`bun run build`
Diff hygiene	`git diff --check`
Visual review	Check new pages at desktop and phone widths
Environment	Confirm secrets, bucket names, and public build arguments
Database	Back up before migrating; review new migration files
Storage	Verify both bucket policies and a real upload in the target environment
After deploy	Open `/health-check`; trigger a sync; confirm last-sync time updates

9.7 Backup and Recovery

- Back up PostgreSQL on a schedule and test restores; the `projects` read model can be rebuilt from ABEMIS, but engagement data and audit logs cannot.
- Back up both MinIO buckets; evidence is not recoverable from other sources.
- Keep `.env.production` secrets in a secure store.
- Use `docs/migrations/*-rollback.sql` for the migrations that have prepared rollbacks.

10. Quality and Testing Design

10.1 Test Layers

Layer	What is covered	Tooling
Unit and policy tests	About 230 test files beside the code: permissions, scope, year scope, stage mapping, transforms, upload policy, chat policy and limits, SQL guardrails, SLA logic, DTO output, analytics queries, parsers	`bun test`
Handler tests	Route handlers built with injectable dependencies (notifications, upload preview, chat) so authorization and error paths can be tested without a server	`bun test`
Static checks	Type safety and lint rules	`tsc --noEmit`, ESLint
Build check	Production build with standalone output	`bun run build`
Manual and browser review	Responsive layout, accessibility, map and chart behavior, real upload against the target storage	Release checklist

This document did not re-run the test suite. The security review of 27 September 2026 recorded a passing type check, most tests passing, and a small number of lint errors and test failures at that time; run the checklist before each release.

10.2 Design for Testability

- Pure functions hold rules (scope conditions, status mapping, SLA thresholds, banner parsing) so they can be tested without a database.
- Route handlers receive their dependencies as arguments, which allows tests to supply fakes.
- Constants (SLA hours, limits) live in one module and are shared by code and reports, so tests and behavior cannot drift apart.

10.3 Acceptance

Acceptance criteria for each requirement are in the RAD traceability matrix (RAD Section 8.3). A release is accepted when the checklist in Section 9.6 passes and the criteria for changed requirements are demonstrated.

11. Appendices

Appendix A. Environment Variable Reference

Names only; values are secrets or deployment-specific. Variables prefixed `NEXT_PUBLIC_` are inlined at build time.

Group	Variables
Database	`DATABASE_URL`, `DATABASE_POOL_MAX`, `DATABASE_IDLE_TIMEOUT`, `DATABASE_MAX_LIFETIME`, `RUN_DB_MIGRATIONS`, `BASELINE_EXISTING_DATABASE`, `DRIZZLE_INCLUDE_AUTH`
Authentication	`BETTER_AUTH_SECRET`, `BETTER_AUTH_URL`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `EMAIL_OTP_EXPIRY`, `ADMIN_EMAIL`, `ADMIN_EMAILS`, `ADMIN_NAME`, `ADMIN_PASSWORD`
Application	`NEXT_PUBLIC_APP_URL`, `NEXT_PUBLIC_BASE_URL`, `APP_PORT`, `DOCKER_NETWORK`, `ENABLE_SCHEDULER`, `ENABLE_BACKGROUND_WORKER`
ABEMIS and sync	`ABEMIS_BASE_URL`, `ABEMIS_API_KEY`, `ABEMIS_INFRA_ENDPOINT`, `SYNC_CRON_SCHEDULE`
Object storage	`MINIO_ENDPOINT`, `MINIO_PORT`, `MINIO_USE_SSL`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `MINIO_BUCKET_NAME`, `MINIO_PRIVATE_BUCKET_NAME`, `NEXT_PUBLIC_MINIO_ENDPOINT`, `NEXT_PUBLIC_MINIO_BUCKET`, `NEXT_PUBLIC_MINIO_USE_SSL`
Uploads and moderation	`ALLOWED_IMAGE_TYPES`, `ALLOWED_VIDEO_TYPES`, `MAX_IMAGE_SIZE`, `MAX_VIDEO_SIZE`, `NSFW_IMAGE_MODERATION_ENABLED`, `NSFW_FAIL_CLOSED`, `NSFW_PORN_THRESHOLD`, `NSFW_HENTAI_THRESHOLD`, `NSFW_SEXY_THRESHOLD`, `FFMPEG_PATH`, `FFPROBE_PATH`
Email	`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_ENCRYPTION`, `MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`
SMS	`SMS_API_URL`, `SMS_SENDER_ID`, `SMS_USERNAME`, `SMS_PASSWORD`, `SMS_GRIEVANCE_API_URL`, `SMS_AUTO_ACK_ENABLED`, `SMS_AUTO_ACK_SINCE`
AI	`AI_PROVIDER`, `AI_MODEL`, `GEMINI_API_KEY`, `GOOGLE_GENERATIVE_AI_API_KEY`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `KIMI_API_KEY`, `ENABLE_MANAGERIAL_AI`
Assistant limits	`CHAT_REQUIRE_AUTH`, `CHAT_RATE_LIMIT_PER_MINUTE`, `CHAT_GLOBAL_DAILY_LIMIT`, `CHAT_HISTORY_RETENTION_DAYS`, `CHAT_RESPONSE_TIMEOUT_MS`, `CHAT_TRUST_PROXY`, `CHAT_RATE_LIMIT_SECRET`
Knowledge base	`KB_CHUNK_SIZE`, `KB_CHUNK_OVERLAP`
Voice	`VOICE_ASSISTANT_ENABLED`, `WAKE_WORD`, `WAKE_WORD_MODEL_PATH`, `WAKE_WORD_THRESHOLD`, `WAKE_WORD_TOKEN_SECRET`, `WAKE_WORD_WS_URL`, `SLEEP_WORD_MODEL_PATH`, `LOCAL_WHISPER_URL`, `LOCAL_WHISPER_TOKEN`, `KOKORO_MODEL_PATH`, `KOKORO_VOICE`
Cache and analytics	`REDIS_URL`, `NEXT_PUBLIC_GA_MEASUREMENT_ID`, `ANALYTICS_PROXY_SECRET`, `ANALYTICS_TRUST_PROXY_HEADERS`
Maps	`NEXT_PUBLIC_GEOSERVER_URL`, `NEXT_PUBLIC_GEOSERVER_WORKSPACE`

Appendix B. Table Catalog

Generated from the live schema on the document date: column and secondary-index counts per table.

Table	Columns	Indexes	Purpose
`account`	13	0	Credential and OAuth links
`account_preferences`	5	0	Notification channel choices
`ai_chat_history`	20	6	Assistant conversation turns

Table	Columns	Indexes	Purpose
ai_chat_rate_limits	4	1	Assistant request windows
analytics_daily_aggregates	9	2	Daily usage rollups
analytics_events	9	3	Minimal usage events
announcements	9	0	Dated announcements
articles	15	1	News and updates
audit_logs	13	4	Audit trail
background_jobs	16	2	Job queue
contact_messages	12	3	Contact and suggestions
contact_responses	7	1	Replies to contact messages
email_templates	11	0	Editable emails
feedback	21	3	Project feedback
feedback_comment_votes	5	1	Comment votes
feedback_comments	15	3	Comments on feedback
issue_responses	13	2	Staff responses to E-Reports
issues	35	3	E-Reports
kb_chunks	7	2	Knowledge chunks and embeddings
kb_documents	20	5	Knowledge documents and FAQs
live_videos	21	2	Broadcast entries
map_layers	15	0	GeoServer layer definitions
mydas_dashboards	6	1	Saved MYDAS dashboards
notification_reads	4	2	Read receipts
notification_recipients	4	3	Per-user notifications
notifications	6	1	Notification events
post_subscriptions	6	2	Post follow subscriptions
project_commodity_overrides	6	0	Commodity overrides
project_data_corrections	9	2	Recorded corrections
project_metric_snapshots	18	4	Daily project snapshots
projects	57	5	ABEMIS project read model
psgc_locations	28	0	PSGC reference data
regional_statistics	17	1	Regional statistics
saved_posts	5	2	Bookmarks
session	9	0	Sessions
sms_grievance_reviews	6	1	SMS review state
spatial_ref_sys	5	0	PostGIS reference
submission_surveys	11	4	Optional surveys
sync_logs	15	0	Sync run history
system_settings	6	0	Key-value settings
upload_sessions	10	1	Resumable uploads
user	14	0	Accounts and roles

Table	Columns	Indexes	Purpose
`user_tour_progress`	4	0	Guided tour progress
`verification`	6	0	One-time codes

Appendix C. Glossary

Term	Meaning
Read model	A copy of source data kept for fast, safe reading
Scope	Region and program assignment that limits a moderator's view
Public DTO	A purpose-built object that contains only fields safe to show publicly
Auto-accept	System setting that publishes feedback and starts issues at `reviewing` after automated checks
Lease	A time-limited claim on a background job that lets another worker take over if the first stops
Dedupe key	Value that prevents two active jobs of the same kind
ANIA	The INFRA Watch AI assistant
MYDAS	Natural-language dashboard designer for administrators

Appendix D. Change History and Document Control

Version	Date	Author	Description
1.0	October 2026	INFRA Watch Project Team	First formal SDD, written from the implemented system and verified against the repository.

Item	Detail
Document	Software Design Document (SDD), INFRA Watch
Version and date	1.0, October 2026
Prepared by	INFRA Watch Project Team
Institution	Department of Agriculture - Bureau of Agricultural and Fisheries Engineering (DA-BAFE)
Companion document	Requirements Analysis Document (RAD), INFRA Watch
Review and approval	Reviewed by: _____ Approved by: _____ Date: _____